# Replenishing perishables on shared delivery routes: Stock-age-based inventory policies, stochastic lookahead and decision-focused learning

**Abstract**

For a periodic-review perishable inventory-routing system with contextual forecasts, FIFO issuing, lost sales and shelf capacity, we propose decision-focused learning (DFL) at two levels: on top of a deliver-up-to-level rule, learning context- and stock-age-dependent order-up-to levels from the realised cost of whole episodes (UL-DFL), and on top of a rolling-horizon inventory-routing model, learning its planning targets (LA-DFL). Twelve policies are compared in closed loop with identical information, from base-stock rules and the rule of Crama et al. (2018) to deterministic, robust and scenario-based lookahead (Jia et al., 2025). We show that the inventory recourse admits FIFO-optimal solutions once early disposal is allowed and that DFL stationary points reduce to the perishable newsvendor fractile. Stock-age-based rules with optimally routed orders are 2–12% cheaper than every integrated policy in twelve of thirteen settings; with expensive routing, a lookahead with day-specific safety quantiles matches them. On fresh test episodes, UL-DFL is 1.1% cheaper than its rule re-tuned on a fine grid and wastes 28% less at nearly equal fill rate, which no parameterisation of the rule achieves. Across thirteen settings it is significantly cheaper than the rule in nine and wastes less in twelve, but on observed fresh-food demand its corrections did not transfer to new products (+0.6%). Without the rule, the same learner is significantly costlier in twelve settings, even with order-up-to structure. LA-DFL improves the deterministic lookahead by up to 5% by learning that only the implemented day needs a high safety quantile, a gain that a day-specific quantile level reproduces.

**Keywords:** Perishable inventory; Inventory routing; Waste; Lost sales; Base-stock policies; Decision-focused learning.

# 1 Introduction

Fresh food is replenished daily, its stock ages every night, and whatever is not sold within its shelf life is thrown away. Retailers and their distributors therefore face the classical perishable inventory trade-off between waste and lost sales [4, 46], but with two complications that single-store inventory models do not capture. First, deliveries to different stores share a small fleet: a store that is not on today's route cannot be replenished today, and a vehicle that is full cannot top up the next store. Second, retailers now have rich contextual information—promotion calendars, weather forecasts, recent sales—that machine-learning forecasters turn into store-day demand predictions. The operational question is how such forecasts should be translated into replenishment and delivery decisions when stock ages, shelf space is limited and vehicles are shared.

Two literatures answer different halves of this question. Perishable inventory control offers simple, robust ordering rules—base-stock policies [12], and rules that anticipate the waste of the current stock [11, 27]—but treats each store in isolation. Inventory routing integrates replenishment and routing, with exact methods for deterministic perishable demand [1, 14], decomposition and order-up-to rules for stochastic perishable demand [17], robust formulations [28, 51] and, recently, scenario-based predict-then-optimise policies that use machine-learning quantile forecasts [37]. In parallel, decision-focused learning (DFL) trains forecasters on the cost of the decisions they induce rather than on forecast accuracy [20, 42]. Whether DFL adds anything once the strong inventory rules and stochastic lookahead methods are on the table has not been tested for perishable inventory routing.

This paper studies a periodic-review perishable inventory-routing system with contextual demand, lost sales, FIFO issuing and shelf capacity, and compares twelve replenishment policies in closed loop, every day observing the age-structured stock and the same forecasts. The comparison includes the inventory-control rules a perishable-inventory manager would use (base-stock with and without estimated waste), the deliver-up-to-level rule of Crama et al. [17], and rolling-horizon inventory-routing models driven by mean, quantile, robust and scenario forecasts, the last being the Scenario Predict-then-Optimize method of Jia et al. [37]. We apply decision-focused learning at two levels. UL-DFL keeps the visit trigger of the deliver-up-to-level rule and learns how far each store's order-up-to level should deviate from the forecast quantile, depending on context and on the age of the store's stock; the learning signal is the realised holding, waste and lost-sales cost of whole training episodes, differentiated through the FIFO dynamics. LA-DFL keeps a deterministic rolling-horizon plan and learns its planning targets, differentiating the realised cost of the plan's deliveries through a coverage layer while an exact MILP oracle fixes the routes.

The answer is that learning pays when it is placed on the right structure, and only when what it learns carries over to the episodes on which the policy is used. Across the settings studied, the stock-age-based rules with their orders routed optimally were cheaper than every integrated policy unless travel was expensive, in which case a lookahead with day-specific safety quantiles matched them. Against a deliver-up-to-level rule re-tuned on a fine grid, UL-DFL was 1.1% cheaper on fresh test episodes of the base case, which played no role in the design, tuning or training of any policy, and it wasted 28% less at almost the same fill rate; no parameterisation of the rule reaches that waste at that fill rate. Across thirteen settings it was significantly cheaper than the rule in nine, by about 1%, and wasted less in twelve. The gain is smallest where the rule already wastes little or where demand is too variable for the context to help, and on observed fresh-food demand the learned corrections did not transfer from the training products to new products, so that the re-tuned rule was cheaper. The gain also depends on the rule: the same learner without the rule, whether it ordered directly or set its own order-up-to levels, was significantly more expensive than UL-DFL in twelve settings and never significantly cheaper than the rule itself. The corrections are interpretable: lower order-up-to levels on promotion days and after the weekly demand peak, when leftover stock meets falling demand. On top of the deterministic lookahead, decision-focused learning revealed that only the executed day needs

a high safety quantile; a lookahead with day-specific quantile levels reproduces that gain without learning.

The paper makes three contributions.

1. **Decision-focused learning of perishable order-up-to levels on shared routes.** UL-DFL learns context- and stock-age-dependent order-up-to levels for a published perishable inventory-routing rule from the realised cost of whole episodes, with model selection on the closed-loop system including exact routing. Starting from the rule re-tuned on a fine grid, it is significantly cheaper than the rule in nine of thirteen settings and on fresh test episodes of the base case, wastes less in twelve, and is the cheapest policy in ten; on observed demand of new products it is not. Its corrections can be read as inventory rules, and an ablation without the rule shows that the structure is what makes the learning pay. We show that the inventory recourse admits FIFO-optimal solutions once early disposal is allowed, bound the marginal value of a delivery by the lost-sales and waste penalties, and prove that stationary points of both decision-focused objectives satisfy the stochastic optimality conditions of the problem with fixed visits, over the whole episode for the order-up-to levels of UL-DFL, reducing to the perishable newsvendor fractile for one-day shelf life; all results are verified numerically.

2. **An inventory-centred benchmark of replenishment policies under routing coupling.** We evaluate stock-age-based inventory rules, a published stochastic PIRP rule, and integrated deterministic, robust, scenario-based and decision-focused lookahead policies under one closed-loop simulator, with every lookahead solved by a MILP with exact route costs to a proven gap (integer routing on the implemented day), three network layouts, 100 test episodes per setting, three training seeds, prespecified hypotheses tested on fresh test episodes, and a case with observed fresh-food demand. Outcomes are reported in inventory terms—cost composition, fill rate, waste rate, delivery frequency and store-level service—across shelf life, cost ratio, fleet tightness, routing cost, demand variability and issuing policy.

3. **Managerial evidence on when integration, stochastic lookahead and learning pay off.** The results show when simple stock-age-based rules suffice and when integration with routing pays, why deterministic plans over-stock and how a day-specific safety stock repairs this, which corrections of an order-up-to rule reduce waste, and how integrated plans concentrate stock-outs at individual stores (Section 6).

Section 2 reviews related work, Section 3 defines the problem and its structural properties, Section 4 describes the policies, Section 5 the experimental design, Section 6 the results, and Section 7 concludes with managerial implications and limitations.

# 2 Related literature

## 2.1 Perishable inventory control

Perishable inventory theory studies ordering when units have a finite lifetime and the age of stock matters for issuing, outdating and service [46]; Bakker et al. [4] and Janssen et al. [36] review the subsequent literature, and Bijvank and Vis [9] review inventory theory with lost sales. Optimal policies depend on the full age vector and are computed by stochastic dynamic programming only for small instances, so practice relies on simple rules and approximations: Chao et al. [13] give approximation algorithms with performance guarantees, and Minner and Transchel [43] derive periodic-review policies under service-level constraints. Base-stock policies that raise the total inventory to a target are the default in automated retail ordering [55], their expected outdating can be approximated from a few product and policy characteristics [56], and Bu et al. [12] show that they become asymptotically optimal when lifetime, demand volume or the penalty costs grow. Stock-age information helps when lifetimes are short. Broekmeulen and

van Donselaar [11] propose to add the estimated waste of current stock to the base-stock order (EWA), Haijema [25] studies optimal ordering, issuing and disposal decisions, Haijema and Minner [26] compare stock-level dependent hybrid base-stock and constant-order rules, and Haijema and Minner [27] show in 11,177 instances that a base-stock policy with an estimated-waste correction (BSP-low-EW) performs close to the SDP optimum and better than the other practical rules. Deep reinforcement learning has been applied to perishable inventory control [10, 18, 54], matches strong heuristics in single-location settings [23], and has been extended to one-warehouse multi-retailer systems [38]. Data-driven newsvendor work uses features to set perishable order quantities directly [5, 34], also when demand observations are censored by stock-outs [48]. All of these studies consider a single stocking point or couple stores only through upstream stock; deliveries do not compete for vehicles and the cost of a delivery does not depend on which other stores are visited.

### 2.2 Perishable inventory routing

Inventory routing integrates replenishment with vehicle routing [16]; with stochastic demand, Kleywegt et al. [40] formulate it as a Markov decision process and Bertazzi et al. [6] study order-up-to policies with stock-outs. For perishables, Federgruen et al. [22] combine the allocation of a perishable product among locations with vehicle routing, Hemmelmayr et al. [30] compare delivery strategies for blood products to hospitals, Coelho and Laporte [14] and Alvarez et al. [1] develop exact formulations and branch-and-cut for deterministic demand with age-dependent inventory, Mirzaei and Seifi [44] include lost sales, and Soysal et al. [52] add demand uncertainty and emissions in a chance-constrained model. Crama et al. [17] study stochastic perishable inventory routing and compare an expected-value policy, the deliver-up-to-level rule $\mathrm{UL}_\lambda$, and decomposition methods; they find that the simple rule is competitive in some settings and that more advanced methods pay off in others. He et al. [28] study a robust perishable IRP under demand uncertainty and solve robust-counterpart reformulations by Benders decomposition and an iterated local search. For non-perishable IRPs under uncertainty, Solyalı et al. [51] derive robust counterparts, Coelho et al. [15] show that rolling-horizon policies using stochastic information outperform deterministic ones, and Malicki and Minner [41] integrate dynamic safety stocks into cyclic inventory routing under non-stationary demand. Most closely related to our question, Jia et al. [37] propose Scenario Predict-then-Optimize: quantile neural forecasts generate demand scenarios for a stochastic rolling-horizon IRP, and the single-stage scenario model performs best on synthetic and industrial data; their model has backlogging and no product ageing. Greif et al. [24] learn the parameters of a prize-collecting TSP policy for dynamic IRP with machine learning. None of these works compares stock-age-based inventory rules, robust and scenario-based lookahead and decision-focused learning under one closed-loop simulator with perishability.

### 2.3 Contextual and decision-focused optimisation

When demand forecasts feed an optimisation model, one can estimate first and then optimise, or train the forecaster on the downstream cost [20, 49]. Decision-focused learning (DFL) differentiates through the optimisation problem, either exactly for convex problems [3, 19] or through surrogates and perturbations for combinatorial ones [7, 42, 53]; solution caching reduces the number of solver calls [45]. The theory is nuanced: under a well-specified model, estimate-then-optimise can converge faster and stochastically dominate integrated training [21, 32], so gains from DFL should be expected mainly when the downstream model is a misspecified approximation. In inventory control, Alvo et al. [2] train neural ordering policies by differentiating simulated costs through known inventory dynamics. For inventory routing, an early DFL prototype exists only as a workshop abstract without perishability [35]. We use the same principle on a perishable inventory-routing rule: UL-DFL differentiates the realised holding, waste and lost-sales cost of

whole episodes through the FIFO dynamics of the deliver-up-to-level rule of Crama et al. [17], so that learning corrects the rule's order-up-to level rather than replacing the rule. A second learner, LA-DFL, differentiates the same cost through a deterministic rolling-horizon PIRP whose routes are fixed by an exact MILP, so that learning corrects the safety stock embedded in the plan rather than the routing decisions.

### 2.4 Positioning

Table 1 positions the study. It is the first, to our knowledge, to evaluate decision-focused learning for perishable inventory routing against the inventory-control rules recommended in the perishable inventory literature and against robust and scenario-based lookahead methods, under identical information and an exact optimisation layer. The methodological novelty is decision-focused learning of the order-up-to levels of a published perishable inventory-routing rule (UL-DFL), complemented by a decision-focused target layer for the rolling-horizon lookahead (LA-DFL) with its structural justification; the managerial contribution is evidence on when integration, stochastic lookahead and decision-focused learning are worth their cost.

Table 1: Positioning relative to the closest studies.

| Study | Ageing | Uncertain demand | Routing | Context | Closed loop | Methods compared |
|---|---|---|---|---|---|---|
| Haijema and Minner [27] | ✓ | ✓ | – | – | ✓ | Stock-age rules vs SDP |
| Broekmeulen and van Donselaar [11] | ✓ | ✓ | – | – | ✓ | EWA vs base-stock |
| Coelho and Laporte [14]; Alvarez et al. [1] | ✓ | – | ✓ | – | – | Exact MILP |
| Crama et al. [17] | ✓ | ✓ | ✓ | – | ✓ | EV, $\mathrm{UL}_\lambda$, decomposition |
| He et al. [28] | ✓ | ✓ | ✓ | – | – | Robust, Benders, ILS |
| Solyalı et al. [51] | – | ✓ | ✓ | – | – | Robust MILP |
| Jia et al. [37] | – | ✓ | ✓ | ✓ | ✓ | Scenario PTO vs PTO, SAA, DFL |
| Islam Sohan and Wasi [35] | – | ✓ | ✓ | ✓ | – | DFL prototype |
| **This study** | ✓ | ✓ | ✓ | ✓ | ✓ | Stock-age rules, $\mathrm{UL}_\lambda$, robust, scenario PTO, DFL |

## 3 Problem description

### 3.1 The periodic-review perishable inventory-routing system

A distributor replenishes a single fresh product at $n$ retail stores $\mathcal{N} = \{1, \ldots, n\}$ from a depot 0. Every morning (day $t$) it decides how much to deliver to each store and how to route a homogeneous fleet of $K$ vehicles of capacity $Q$; each vehicle performs at most one route per day and each store is visited at most once per day. Deliveries arrive before the store opens (zero lead time), as is common for daily fresh-food replenishment. Products arrive fresh and can be sold on $m$ consecutive days, the day of delivery included. Store $i$ can hold at most $C_i$ units at opening (shelf and backroom capacity).

**State and stock age.** Let $B_{irt}$ denote the units at store $i$ at the opening of day $t$, before the delivery, that can still be sold on $r$ days including day $t$ ($r = 1, \ldots, m-1$). The vector $B_t = (B_{irt})$ is the age-structured inventory state observed by the planner. Units with $r = 1$ are on their last day.

**Daily dynamics.** Given deliveries $q_t = (q_{it})$ and realised demand $D_{it}$, the following happens at each store. (i) Available stock by remaining life is $S_{irt} = B_{irt}$ for $r < m$ and $S_{imt} = q_{it}$. (ii) Demand is served first-in first-out (FIFO), oldest units first. (iii) Unmet demand is lost. (iv) Unsold units with $r = 1$ expire. (v) Remaining units are carried overnight and lose one

day of remaining life. With sales $w_{irt} = \min\{S_{irt}, (D_{it} - \sum_{r'<r} w_{ir't})^+\}$, lost sales are $u_{it} = D_{it} - \sum_r w_{irt}$, expired units are $e_{it} = S_{i1t} - w_{i1t}$, and the next state is $B_{i,r,t+1} = S_{i,r+1,t} - w_{i,r+1,t}$ for $r = 1, \dots, m-1$. As a robustness check we also simulate partial last-in first-out (LIFO) issuing, in which a fraction of customers picks the freshest units [25].

**Costs.** Day $t$ costs

$$c_t = c^{\mathrm{r}}\,\mathrm{TSP}(\mathrm{routes}_t) + h \sum_{i,r} B_{i,r,t+1} + p_s \sum_i e_{it} + p_u \sum_i u_{it}, \tag{1}$$

where $\mathrm{TSP}(\cdot)$ is the travelled distance, $h$ the holding cost per unit carried overnight, $p_s$ the outdating (waste) cost per expired unit and $p_u$ the lost-sales penalty. Throughout, $h \le p_s < p_u$: carrying a unit one more night is cheaper than discarding it, and a lost sale is the most expensive event.

**Information and policies.** Before deciding on day $t$ the planner observes the state $B_t$, the demand history up to day $t-1$, and contextual information about the next days: calendar, announced promotions and holidays, and a weather forecast. A *policy* maps this information to deliveries and routes. The objective is to minimise the expected cost $\mathbb{E}\big[\sum_{t=1}^{T} c_t\big]$ over a horizon of $T$ days. Delivery decisions couple days through stock age and stores through vehicle capacity and route costs; this is what distinguishes the problem from single-store perishable inventory control [27, 46] and makes it a stochastic perishable inventory-routing problem [17].

### 3.2 Lookahead model

All integrated policies in Section 4 solve, every day, an $H$-day lookahead model and implement only its first day (rolling horizon). The model takes the observed state $B_t$ and one or several planning demand paths $\tilde{d}^s = (\tilde{d}^s_{ik})_{i\in\mathcal{N},k=0..H-1}$, $s \in \mathcal{S}$, as inputs. Its routing part uses a vehicle-assignment formulation with exact route costs. Let $\mathcal{R} = 2^{\mathcal{N}}$ be the set of store subsets and $\mathrm{TSP}(R)$ the length of an optimal depot-to-depot tour through $R$, precomputed by the Held–Karp recursion [29]. Binary $z_{ivk}$ assigns store $i$ to vehicle $v$ on lookahead day $k$; continuous $y_{Rvk} \ge 0$ selects the route of vehicle $v$; $g_{ivk} \ge 0$ is the load for store $i$ on vehicle $v$; $q_{ik} = \sum_v g_{ivk}$.

$$\begin{aligned}
\min \sum_{k,v,R} c^{\mathrm{r}}\mathrm{TSP}(R)\, y_{Rvk} + \frac{1}{|\mathcal{S}|} \sum_{s\in\mathcal{S}} \sum_{k,i} \Big( & h \sum_{r\ge 2} (S^s_{irk} - w^s_{irk} - \zeta^s_{irk}) \\
& + p_s (S^s_{i1k} - w^s_{i1k}) + p_s \sum_{r\ge 2} \zeta^s_{irk} + p_u u^s_{ik} \Big) \qquad (2)
\end{aligned}$$

$$\text{s.t.} \quad \textstyle\sum_R y_{Rvk} = 1, \quad \sum_{R\ni i} y_{Rvk} = z_{ivk}, \quad \sum_v z_{ivk} \le 1 \qquad \forall i, v, k, \tag{3}$$

$$g_{ivk} \le \min\{Q, C_i\}\, z_{ivk}, \quad \textstyle\sum_i g_{ivk} \le Q \qquad \forall i, v, k, \tag{4}$$

$$S^s_{imk} = q_{ik}, \quad S^s_{ir0} = B_{irt}\ (r < m) \qquad \forall i, r, k, s,$$

$$S^s_{i,r,k+1} = S^s_{i,r+1,k} - w^s_{i,r+1,k} - \zeta^s_{i,r+1,k} \qquad \forall i, r, k, s, \tag{5}$$

$$\textstyle\sum_r w^s_{irk} + u^s_{ik} = \tilde{d}^s_{ik}, \quad w^s_{irk} + \zeta^s_{irk} \le S^s_{irk}, \quad \sum_r (S^s_{irk} - \zeta^s_{irk}) \le C_i \qquad \forall i, r, k, s, \tag{6}$$

with all variables non-negative and $z$ binary. Constraint (3) lets each vehicle choose one route (the empty set has zero cost) whose store set equals the stores assigned to it; (4) enforces vehicle and shelf capacity; (5) is the age-structured balance; (6) links sales, lost sales and the shelf capacity at opening. $\zeta$ is an excess-disposal slack priced at $p_s$; it is only needed when $|\mathcal{S}| > 1$ (in a deterministic plan stock never exceeds what was planned) and it is never used on day 0, where the state is known. With $|\mathcal{S}| = 1$ the model is a deterministic rolling-horizon PIRP; with $|\mathcal{S}| > 1$ it is the single-stage scenario model (one routing and delivery plan for all scenarios) that Jia et al. [37] found to perform best. A symmetry-breaking constraint assigns the lowest-indexed visited store to vehicle 1.

**Proposition 1** (Exact routing cost)**.** *For every binary $z$ satisfying (3), the only feasible $y$ sets $y_{R_{vk},v,k} = 1$ for $R_{vk} = \{i : z_{ivk} = 1\}$. Hence the routing term of (2) equals the exact travelled distance of the assigned routes, and the model is an exact MILP formulation of the $H$-day lookahead PIRP in which only the $nKH$ assignment variables are integer.*

The proof is in Appendix A. Proposition 1 matters computationally: with ten stores the model has $2^{10}KH$ continuous route columns but only $nKH$ binaries, whereas an arc-based vehicle-indexed formulation could not be solved within the time budget in preliminary experiments. Because only day 0 of each plan is implemented, the experiments impose integrality on the day-0 assignment and relax it on later lookahead days; the relaxed routing term is then a convex lower estimate of future travel cost. With this choice HiGHS [33] proves a gap of at most 0.5% for every decision within the time limit (Section 6.8); in a pilot on four episodes, imposing integrality on all lookahead days changed episode costs by between $-1.5\%$ and $+3.3\%$ at three to five times the solution time and with time-limit stops. Day-0 routes of every implemented decision are additionally re-optimised by exact enumeration of all feasible route sets.

### 3.3 Structural properties

The results below connect the lookahead model with the physical FIFO system and justify the learning signal used by both learners (Sections 4.3 and 4.5). Fix one store, a delivery plan $q = (q_0, \dots, q_{H-1})$, an initial state $B$ and a demand path $d$. Let $C_{\mathrm{FIFO}}(q; d)$ be the inventory cost (holding, waste and lost sales) of the physical system above, and let $V_{\mathrm{LP}}(q; d)$ be the optimal value of the *recourse LP*: constraints (5)–(6) for one scenario with $q$ fixed, the issuing order left free and early disposal $\zeta$ allowed.

**Theorem 1** (Complete recourse and FIFO optimality)**.** *(i) For any state with $\sum_r B_{ir} \le C_i$ and any non-negative planning demand, the lookahead MILP is feasible and its optimal value is finite. (ii) For fixed deliveries, the recourse LP has an optimal solution in which every day's sales follow FIFO and demand is served whenever stock is available. (iii) $V_{\mathrm{LP}}(\cdot; d)$ is convex and piecewise linear, and $V_{\mathrm{LP}}(q; d) \le C_{\mathrm{FIFO}}(q; d)$, with equality whenever early disposal is not profitable along the FIFO trajectory.*

The disposal option in part (ii) cannot be dropped: without it, age-increasing holding costs do not make every optimal solution FIFO. If an expiring unit and a fresh unit are on the shelf and only one more unit will ever be sold, selling the fresh unit and letting the old one expire saves the fresh unit's holding cost. Allowing disposal at cost $p_s$ restores the exchange argument (the younger unit can always replicate the older unit's path) and makes the recourse a convex relaxation of the physical system.

**Proposition 2** (Marginal value of a delivery)**.** *(a) The dual multiplier $\pi_k$ of the fresh-stock constraint $S_{imk} = q_k$ is a subgradient of $V_{\mathrm{LP}}$ with respect to $q_k$, and $-p_u \le \pi_k \le p_s$. (b) If $h \le p_s$, $C_{\mathrm{FIFO}}(\cdot; d)$ is piecewise linear and Lipschitz, and its one-sided partial derivatives satisfy*

$$-p_u \;\le\; \frac{\partial C_{\mathrm{FIFO}}}{\partial q_k} \;\le\; p_s + (H - 1 - k)\, h. \tag{7}$$

Proposition 2 states the familiar newsvendor logic in a multi-period, age-structured form: an extra delivered unit can at most prevent one lost sale, and at worst it is carried to the end of the horizon or wasted (possibly by displacing a later delivery). The asymmetry $p_u \gg p_s$ is what makes point forecasts a poor input for perishable replenishment. The bound in (b) is tight (Section 6.9).

Both learners of Section 4 change deliveries only through a quantity they control: LA-DFL through the planning targets of the lookahead (Section 4.5), UL-DFL through the order-up-to levels of the rule (Section 4.3). In the lookahead, for a fixed visit pattern, a delivery on day $k$

covers the planned demand of the days until the next visit, limited by shelf life and horizon, net of the stock that will still be usable. Let $W_k$ denote this coverage window. For the rule, let $C^T$ be the holding, waste and lost-sales cost of a whole episode of $T$ days, and $U_{it}$ the usable stock of store $i$ on day $t$ (its stock net of the units projected to expire).

**Proposition 3** (Decision-focused stationarity)**.** *(a)* Planning targets (LA-DFL). *Fix the visit pattern and consider planning targets $\tilde{d}_j$ at which no capacity binds and the coverage map is differentiable. Then $\partial q_k / \partial \tilde{d}_j \in \{0, 1\}$, with value one exactly when day $j$ lies in the coverage window $W_k$ and is not already covered by planned stock, and for such $j$*

$$\frac{\partial}{\partial \tilde{d}_j} \mathbb{E}\big[C_{\mathrm{FIFO}}(q(\tilde{d}); D) \mid \xi\big] = \mathbb{E}\Big[\frac{\partial C_{\mathrm{FIFO}}}{\partial q_{k(j)}} \,\Big|\, \xi\Big], \tag{8}$$

*where $k(j)$ is the visit covering day $j$. A stationary point of the decision-focused objective therefore satisfies the first-order optimality condition of the stochastic program in which only the delivery quantities of the fixed visit pattern are optimised. (b)* Order-up-to levels (UL-DFL). *Fix the store-days on which the visit trigger fires and consider order-up-to levels $\tilde{S}_{it}$ at which no shelf or fleet capacity binds. Then a triggered delivery is $q_{it} = \tilde{S}_{it} - U_{it}$, so $\partial q_{it} / \partial \tilde{S}_{it} = 1$, and*

$$\frac{\partial}{\partial \tilde{S}_{it}} \mathbb{E}\big[C^T(\tilde{S}; D) \mid \phi_{it}\big] = \mathbb{E}\Big[\frac{\mathrm{d}C^T}{\mathrm{d}q_{it}} \,\Big|\, \phi_{it}\Big], \tag{9}$$

*the expected total effect of one more delivered unit on the cost of the episode, including its effect on later deliveries through the stock it leaves. If the correction $\delta_\theta$ can take any value for each input $\phi_{it}$, a stationary point therefore satisfies the first-order optimality condition of the stochastic program in which the order-up-to levels of the triggered store-days are optimised for the whole episode. (c) With a single-day shelf life, (8) and (9) vanish exactly at the perishable newsvendor quantile, $F_{D|\xi}(\tilde{d}) = p_u/(p_u + p_s)$ and $F_{D|\phi}(\tilde{S}) = p_u/(p_u + p_s)$.*

Proposition 3 clarifies what decision-focused training can and cannot do. For the lookahead, it can correct the safety stock embedded in deterministic plans, store by store and context by context, which is where the cost asymmetry and stock age act; it does not optimise visit days, which are left to the MILP. For the rule, the trigger plays the role of the MILP: training can move the order-up-to level of every triggered store-day towards the level that is optimal over the whole episode for the given visits, which a single service level cannot do (Section 6.3), but it does not change when the rule visits a store. It also explains the relation to the estimate-then-optimise literature: when the forecasting model is well specified and decisions are optimised against the full predictive distribution, integrated training has no first-order advantage [21, 32]; a gain is possible only when the downstream model is a misspecified deterministic approximation, as all rolling-horizon PIRP policies are.

# 4 Replenishment policies

All policies receive the same information each day: the observed age-structured stock $B_t$ and the same contextual forecasts. They differ only in how the forecasts and the stock are turned into deliveries and routes. Table 2 summarises the twelve policies and NB-DFL, a second ablation of UL-DFL (Section 4.3); parameters marked as tuned are selected on validation episodes only. Decision-focused learning is applied at two levels: on top of the deliver-up-to-level rule (UL-DFL, Section 4.3), where it learns the order-up-to level, and on top of the deterministic lookahead (LA-DFL, Section 4.5), where it learns the planning targets.

## 4.1 Shared forecasts

For every store $i$, planning day $t$ and lookahead offset $k \in \{0, \ldots, H-1\}$, one LightGBM model [39] predicts the mean $\hat{\mu}_{i,t+k}$ and thirteen LightGBM quantile models predict the quantiles

$\hat{Q}_\tau(D_{i,t+k} \mid \xi)$ for $\tau \in \{0.05, 0.1, 0.2, \ldots, 0.9, 0.95, 0.975, 0.99\}$ (sorted to avoid crossing). Features are store identity, offset $k$, calendar, announced discount and holiday, temperature forecast, and summaries of the last 1, 7 and 28 days of demand. A forecast scale is $\hat{\sigma} = (\hat{Q}_{0.9} - \hat{Q}_{0.1})/(2z_{0.9})$, and quantiles at other levels are linearly interpolated. Quantile regression of this type is the standard data-driven newsvendor input [5, 34].

### 4.2 Inventory-control rules with route-second delivery

These policies represent current replenishment practice: an inventory rule sets the store orders, and the orders are then routed. Deliveries are routed by an exact capacitated vehicle-routing oracle that enumerates all feasible route sets (possible because $n = 10$). If the fleet cannot carry all orders, the part exceeding today's median demand is reduced proportionally until a feasible packing exists.

**BSP: periodic base-stock.** Stores are served on a fixed cycle of $R \in \{1, 2\}$ days (for $R = 2$ in two alternating groups). On a delivery day, store $i$ is raised to the order-up-to level $S_{it} = \hat{Q}_\alpha(\sum_{k<R} D_{i,t+k})$ (normal approximation for $R > 1$). The inventory position counts all units regardless of age, as in most retail ordering systems [27].

**BSP-EW: base-stock with estimated waste.** Identical to BSP, except that units of the current stock that are expected to expire unsold during the cycle are removed from the inventory position: $q_{it} = (S_{it} - \sum_r B_{irt} + \widehat{EW}_{it})^+$, where $\widehat{EW}_{it}$ is computed by projecting the age profile forward under FIFO with the mean forecasts. This is the "estimated withdrawal and aging" idea of Broekmeulen and van Donselaar [11]; waste-anticipating base-stock rules of this family were found to be near-optimal among practical stock-age-dependent policies by Haijema and Minner [27].

**UL$_\lambda$: deliver-up-to-level rule [17].** A store is visited when its stock does not cover the current day's demand at service level $\beta$, i.e. when $\sum_r B_{irt} < \hat{Q}_\beta(D_{it})$; it then receives enough to reach the $\alpha$-quantile of the demand of the next $\lambda$ days, capped by its shelf room. This is the state-dependent "deliver-up-to-level" rule that Crama et al. [17] identify as a simple policy with good performance in some settings; it acts as an $(R, s, S)$ policy with $R = 1$. Our implementation differs from the original in four disclosed respects. (i) The original uses one target service level for both the trigger and the delivery quantity (90% in its experiments); we tune the two levels separately on validation cost, and the grid contains the original's case $\beta = \alpha$. (ii) The original counts the total stock; here the order-up-to level is reached net of the stock projected to expire within the $\lambda$ days, as in BSP-EW, while the trigger counts the total stock as in the original. (iii) The quantiles come from the shared contextual forecasts instead of a known demand distribution. (iv) The original routes the selected deliveries by solving a vehicle routing problem, in its experiments with a fast LP-based heuristic; here they are routed exactly. The original compares $\lambda = 1$ with $\lambda = m - 1$; we tune $\lambda \in \{1, 2, 3\}$. Appendix B evaluates the rule exactly as published, tuned on the same validation episodes; it is 0.6–3.4% more expensive than our version in every configuration.

### 4.3 Decision-focused order-up-to levels (UL-DFL)

UL-DFL (proposed) keeps the visit trigger and the delivery logic of UL$_\lambda$ and learns how far the order-up-to level of each store-day should deviate from the forecast quantile:

$$\tilde{S}_{it}(\theta) = S_{it} + \hat{s}_{it}\,\delta_\theta(\phi_{it}), \qquad q_{it} = \mathbf{1}\Big[\textstyle\sum_r B_{irt} < \hat{Q}_\beta(D_{it})\Big]\,\Big(\tilde{S}_{it} - \textstyle\sum_r B_{irt} + \widehat{EW}_{it}\Big)^+, \tag{10}$$

where $S_{it} = \hat{Q}_\alpha(\sum_{k<\lambda} D_{i,t+k})$ is the tuned rule's level, $\hat{s}_{it}$ the forecast scale of the $\lambda$-day demand, and $\delta_\theta$ a two-layer network (32 SiLU units per layer) whose output layer is initialised to zero, so that training starts exactly at the tuned $\mathrm{UL}_\lambda$. The input $\phi_{it}$ contains the store's forecast features for day $t$ (discount, promotion and holiday flags, temperature forecast, weekday, recent demand), the scaled mean, scale and 5–95% range of the forecast, and the store's stock by remaining life. A correction $\delta_\theta = 0.5$ raises the level by half a forecast standard deviation; under a normal approximation it moves the implied service level from $\alpha$ to $\Phi(z_\alpha + \delta_\theta)$.

The rule covers only $\lambda$ days and ignores what happens to the stock afterwards, in particular that units delivered today are carried into tomorrow and expire after $m$ days. UL-DFL learns this from the realised cost of whole episodes. On a training episode the rule is simulated for all 21 decision days with the age-structured FIFO dynamics of Section 3.1, written as a differentiable program: sales, waste and carried stock are compositions of minima and differences of the stock vector and the deliveries, shelf capacity caps each delivery, and fleet capacity scales the day's deliveries down proportionally when their total exceeds $KQ$. The visit trigger is held fixed (it is piecewise constant in $\theta$). Back-propagation through time then returns the gradient of the episode's holding, waste and lost-sales cost with respect to $\theta$. Following Proposition 2, and with the stationary points characterised in Proposition 3(b), an extra unit in a delivery is pushed up when, in hindsight, it would have prevented a lost sale on any later day, and pushed down when it would have been carried for several nights or wasted; unlike the lookahead-based learner below, these effects are traced over the whole episode rather than over the $H$ days of a plan. Neural ordering policies trained through simulated inventory dynamics in this way have been proposed for single-location lost-sales systems by Alvo et al. [2]; we use the same principle to correct, rather than replace, a published PIRP rule.

Exact packing, routing and routing cost do not enter the gradient. They enter model selection: after every epoch the current network is run in the full closed-loop simulator on the validation episodes, and the network with the lowest validation cost, which may be the tuned rule itself, is kept. We train three networks from different random seeds and keep the one with the lowest validation cost. Once trained, UL-DFL needs one forward pass per store and day, so it is as fast online as $\mathrm{UL}_\lambda$.

**NN-DFL: the same learner without the rule.** To separate the contribution of the rule's structure from that of learning, NN-DFL uses the same inputs, the same differentiable episode simulation, loss, seeds and validation-based model selection, but none of the structure of $\mathrm{UL}_\lambda$: the network outputs the delivery directly, $q_{it} = \bar{d}_i\,(1 + \delta_\theta(\phi_{it}))^+$ with $\bar{d}_i$ the store's mean daily demand, a store is visited whenever $q_{it} > 0$, and training starts from delivering one day of mean demand to every store every day. This is the type of neural ordering policy trained through simulated inventory dynamics by Alvo et al. [2]. Because it starts far from a good policy, it is given twice as many epochs as UL-DFL. **NB-DFL** separates the two parts of the rule that NN-DFL removes: it keeps the order-up-to structure but not the rule's forecast quantile and visit trigger. The network outputs the order-up-to level, $\tilde{S}_{it} = \bar{d}_i(1 + \delta_\theta(\phi_{it}))^+$, the store receives $(\tilde{S}_{it} - U_{it})^+$ with $U_{it}$ its usable stock, it is visited whenever this is positive, and training starts from a level of one day of mean demand. It is trained like NN-DFL.

### 4.4 Integrated rolling-horizon policies

These policies solve the lookahead MILP of Section 3.2 every day with $H = 3$ ($H = 2$ when $m = 2$) and implement day 0. They differ only in the planning demand.

- **LA-Mean** uses the point forecasts $\hat{\mu}$: the classical predict-then-optimise policy.
- **LA-Quantile** uses $\hat{Q}_\alpha$ for every store and day. With $\alpha$ tuned on validation episodes, this is a parametric cost-function approximation in the sense of Powell [47]: a deterministic lookahead whose inputs are adjusted to perform well under uncertainty.

- **LA-Quantile2** uses $\hat{Q}_{\alpha_0}$ on the implemented day and $\hat{Q}_{\alpha_1}$ on all later lookahead days, with $(\alpha_0, \alpha_1)$ tuned. It is an ablation of LA-DFL: it can represent the day-specific safety stock that LA-DFL learns (Section 6.2) but none of its context- or stock-dependent corrections.

- **LA-Robust** uses budgeted demand uncertainty [8]. Demand on lookahead day $k$ ranges over $[\hat{\mu}_k, \hat{\mu}_k + \hat{\delta}_k]$ with $\hat{\delta}_k = \hat{Q}_{0.95} - \hat{\mu}_k$, and the cumulative scaled deviation of the first $k+1$ days is at most $\Gamma_k = \min\{\Gamma\sqrt{k+1}, k+1\}$. The planning path reproduces the worst-case cumulative demand of every prefix, which is the deterministic equivalent shown by Bertsimas and Thiele [8] for inventory systems and used in robust inventory routing [51]. $\Gamma$ is tuned.

- **ScenPO-SS** is the Scenario Predict-then-Optimize method of Jia et al. [37], with its scenario set: the predicted deciles $0.1, \ldots, 0.9$ of every store-day are themselves scenarios, and eleven further scenarios each draw one band between neighbouring deciles uniformly and then every store-day value uniformly inside that band, giving 20 equally weighted scenarios as in the original study. The single-stage scenario model chooses one delivery and routing plan that minimises the average cost over the scenarios. Two adaptations are needed and disclosed: inventory follows the perishable age-structured dynamics with lost sales instead of the original non-perishable backlogging model, and the quantile model is LightGBM rather than an MQ-RNN, so that all policies use the same forecasts. Appendix B reports a variant that samples ten scenarios from bands of the full quantile grid.

- **LA-DFL** uses decision-focused planning targets (Section 4.5); in this study it serves mainly as a diagnostic of the deterministic lookahead.

- **PI** solves the same lookahead with the realised demand of the next $H$ days. It is a value-of-information reference, not a lower bound.

Table 2: Policies compared. All use the same observed stock and the same forecasts.

| Policy | Family | Decision rule | Tuned on validation |
|---|---|---|---|
| BSP | Inventory rule, route second | Periodic base-stock on total stock | Cycle $R$, quantile $\alpha$ |
| BSP-EW | Inventory rule, route second | Base-stock plus estimated waste [11, 27] | $R$, $\alpha$ |
| $\text{UL}_\lambda$ | Stochastic PIRP rule | Deliver-up-to-level [17] | $\lambda$, $\alpha$, $\beta$ |
| UL-DFL | Stochastic PIRP rule | Deliver-up-to-level with decision-focused order-up-to level (this paper) | Network weights, early stopping, seed |
| NN-DFL | Neural ordering policy (ablation) | Delivery output by a network trained like UL-DFL [2] | Network weights, early stopping, seed |
| NB-DFL | Neural base-stock policy (ablation) | Order-up-to level output by a network trained like UL-DFL | Network weights, early stopping, seed |
| LA-Mean | Rolling-horizon PIRP | Mean forecasts | – |
| LA-Quantile | Rolling-horizon PIRP (CFA) | Quantile forecasts | $\alpha$ |
| LA-Quantile2 | Rolling-horizon PIRP (CFA) | Day-specific quantile forecasts | $\alpha_0$, $\alpha_1$ |
| LA-Robust | Rolling-horizon PIRP | Budgeted uncertainty [8] | $\Gamma$ |
| ScenPO-SS | Rolling-horizon stochastic PIRP | Scenario PTO [37] | – (20 scenarios) |
| LA-DFL | Rolling-horizon PIRP | Decision-focused targets (this paper; diagnostic) | Network weights, early stopping, seed |
| PI | Reference | Realised demand | – |

### 4.5 Decision-focused planning targets (LA-DFL)

LA-DFL keeps the deterministic lookahead, which is fast and easy to explain to planners, and learns how much each planning target should deviate from the forecast. Its role in this study is diagnostic: its corrections show where a deterministic plan mis-stocks (Section 6.2), and the ablation LA-Quantile2 tests whether that diagnosis is enough. For store $i$, planning day $t$ and offset $k$ the target is

$$\tilde{d}_{i,t+k}(\theta) = \Big(\hat{Q}_{\alpha^\star}(D_{i,t+k} \mid \xi) + \hat{\sigma}_{i,t+k}\, \delta_\theta(\phi_{itk})\Big)^+, \tag{11}$$

where $\alpha^\star$ is the quantile level selected for LA-Quantile, and $\delta_\theta$ is a two-layer neural network whose output layer is initialised to zero, so that training starts exactly at the tuned LA-Quantile policy. The input $\phi_{itk}$ contains the forecast features, the scaled mean, scale and 5–95% range of the forecast, the offset $k$, and the store's current stock by remaining life. The correction can therefore depend on context (e.g. promotions, which raise both mean and variance) and on stock age, which neither the forecasts nor a single quantile level capture.

**Learning signal.** The visit pattern chosen by the MILP is piecewise constant in the targets, so its gradient carries no information. Given the pattern, however, deliveries are piecewise linear in the targets: each delivery covers the planned demand of its coverage window $W_k$, net of the usable stock projected by FIFO, capped by the remaining shelf room and scaled to the vehicle capacity. We implement this coverage map $q(\tilde{d}; V)$ as a differentiable layer and evaluate the realised $H$-day FIFO cost of the resulting deliveries under the observed demand with a differentiable simulator.

Automatic differentiation then returns

$$\nabla_\theta C_{\mathrm{FIFO}}(q(\tilde{d}_\theta; V), D) = \Big(\frac{\partial \tilde{d}}{\partial \theta}\Big)^\top \Big(\frac{\partial q}{\partial \tilde{d}}\Big)^\top \nabla_q C_{\mathrm{FIFO}}, \tag{12}$$

whose last factor is the marginal value of a delivery bounded in Proposition 2, and whose stationary points are characterised in Proposition 3. In words: if, in hindsight, an extra unit in a store's delivery would have prevented a lost sale, the targets of the days that delivery covered are pushed up; if the extra unit would have been wasted or carried, they are pushed down; and nothing changes when the store was not visited or capacity was binding.

**Training.** Training states are the decision epochs visited by the tuned LA-Quantile policy on training episodes. Each epoch re-solves the MILP at the current targets to refresh the visit patterns (a form of solution caching, 45), takes Adam steps on the average realised cost of minibatches, and evaluates the closed-loop cost of the current network on the validation episodes; the network with the lowest validation cost is kept, which may be the LA-Quantile starting point. Algorithm 1 summarises the procedure.

**Algorithm 1** Training of decision-focused planning targets (LA-DFL)

**Require:** training states $\{(B^{(s)}, \xi^{(s)}, D^{(s)})\}$ from LA-Quantile runs; tuned level $\alpha^\star$; epochs $E$
initialise $\delta_\theta \equiv 0$; $\theta^\star \leftarrow \theta$; $J^\star \leftarrow$ validation cost of LA-Quantile
**for** $e = 1, \dots, E$ **do**
  for every state $s$: solve the lookahead MILP with targets $\tilde{d}_\theta^{(s)}$; store visit pattern $V^{(s)}$
  **for** each minibatch $\mathcal{B}$ **do**
    $L \leftarrow |\mathcal{B}|^{-1} \sum_{s \in \mathcal{B}} C_{\mathrm{FIFO}}\big(q(\tilde{d}_\theta^{(s)}; V^{(s)}), D^{(s)}\big)$ ▷ differentiable coverage layer + FIFO simulator
    $\theta \leftarrow \mathrm{Adam}(\theta, \nabla_\theta L)$
  **end for**
  $J \leftarrow$ closed-loop validation cost of LA-DFL with $\theta$; **if** $J < J^\star$ **then** $\theta^\star \leftarrow \theta$, $J^\star \leftarrow J$
**end for**
**return** $\theta^\star$

The two learners differ in what the gradient sees. LA-DFL learns from the $H$-day cost of one plan with the visit pattern chosen by the MILP, so it can only correct the plan's targets; UL-DFL learns from the cost of whole episodes, so it can trade today's order against tomorrow's carried stock and waste, but it keeps the rule's simple visit trigger. LA-DFL needs an MILP solve per training state and epoch; UL-DFL needs none. Three design choices of LA-DFL matter. The learner does not differentiate a heuristic surrogate of the MILP; the MILP is solved to a proven gap and only supplies visit patterns. The loss is the realised physical cost rather than regret against a full-information plan. And training and model selection are performed on the closed-loop system that is evaluated.

# 5 Experimental design

## 5.1 Network, products and costs

The base network consists of a depot and the first ten customers of Solomon's R101 instance [50], which are scattered around the depot at distances of 11 to 32; coordinates define Euclidean distances and time windows are ignored. Two further layouts use the first ten customers of C101, a tight cluster 15–21 distance units from the depot (*clustered*), and of RC101, a remote cluster of eight stores 33–40 units away plus two stores near the depot (*mixed*). Two vehicles serve the stores. Vehicle capacity is set through the fleet ratio $\rho = KQ/\bar{D}$, where $\bar{D}$ is mean total daily demand; the base case has $\rho = 1.25$ (117 units per vehicle), so that the fleet can carry a quarter more than average demand, as is common in grocery distribution where vehicles are sized for

peak days. Shelf capacity is three days of mean store demand. The base product has a shelf life of $m = 3$ days. Costs are $h = 1$ per unit and night, $p_s = 5$ per expired unit and $p_u = 20$ per lost sale, with routing cost one per distance unit; per unit of demand, waste is five times and a lost sale twenty times as expensive as one night of holding. Only these ratios matter for the policies. The ratio $p_u/(p_u + p_s) = 0.8$ is the critical fractile of the perishable newsvendor (Proposition 3) and is varied through $p_u$ in the sensitivity analysis; $h$ and $p_s$ are kept fixed. The holding cost is high relative to the write-off cost, as is appropriate for fresh products whose overnight storage takes refrigerated space and handling and lowers their quality; a lower holding cost would make carried stock cheaper for every policy.

### 5.2 Demand and information

Demand in the synthetic settings is generated by a contextual negative-binomial model (code in the replication package). Store base levels range from 10 to 22 units per day; demand varies with the day of the week (Saturday +30%), announced promotions (12% of store-days, discounts of 10–30%, multiplying demand by up to 1.8 and lowering the negative-binomial size parameter from 8 to 3.2, so that promotions raise both level and variability), temperature, holidays, and a latent store level following a log-AR(1) process with autocorrelation 0.92 that can only be inferred from recent sales. The coefficient of variation of daily store demand is about 0.4 on regular days. The temperature forecast error grows with the lead time. The model is designed to reproduce qualitative features of fresh retail demand (weekly pattern, promotion lifts, heteroscedasticity, persistence), not calibrated to a specific retailer; the real-data case below tests the policies on observed demand. The forecast features summarise the demand history, which the simulation observes without censoring. In practice sales are censored by stock-outs, which here concern 3–7% of demand under the rules; ordering with censored demand observations is studied by Sachs and Minner [48], and the real-data case reconstructs latent demand from censored sales.

Each experiment uses 200 training, 50 validation and 100 test episodes, generated with disjoint random seeds. An episode consists of 35 days of history (to compute lagged features), 21 decision days, and four further days so that lookahead forecasts and the perfect-information reference are defined on the last decision days. Every policy starts each test episode with empty stores and is evaluated on the same 100 demand paths (common random numbers).

### 5.3 Real-data case

The real-data case replaces the synthetic demand by observed sales of FreshRetailNet-50K [57], a public dataset of 50,000 store–product series of a fresh-food retailer over 97 days (its 90-day training and seven-day evaluation parts combined), with hourly sales, hourly out-of-stock indicators, discounts, holidays and weather (CC BY 4.0 licence). An episode is one product sold at ten stores of the same city over the last 60 days of the data (35 history, 21 decision and four further days). We keep series without missing days and with fewer than 10% zero-sales days and form at most one ten-store group per city and product. Products, not groups, are assigned to the training, validation and test sets, so that no product appears in two of them: the 200 training, 50 validation and 100 test episodes come from 101, 28 and 56 products in 13 cities.

Three preparation steps are needed and disclosed. First, sales are censored on days with stock-outs, which concern about 40% of the store-days. Latent demand is estimated by dividing a day's sales by the share of the series' typical intraday sales profile that falls into in-stock hours, the profile being estimated on the series' fully in-stock days and the share floored at 0.25. Second, the products differ in volume by orders of magnitude, whereas the network, fleet and shelf capacities are those of the synthetic study; each store's latent demand is therefore rescaled to the store's base volume using the mean over the 35 history days only, so that no information from the decision days is used. Third, fleet and shelf capacities are dimensioned on the network's average volume, so groups whose sales level changes by more than $-20\%/+25\%$ between the

history and decision windows (typically products entering or leaving their season) are excluded; in them the fleet, not the replenishment policy, would determine the outcome. This selection concerns only which data enter the experiment; forecasts and policies use past information only. In the resulting test episodes demand on the decision days is on average 6% above the history level, the coefficient of variation of daily store demand is 0.55, and discounts are offered on 44% of the store-days. Forecast features are those of the synthetic study, with the observed temperature used as its own forecast.

### 5.4 Configurations

Besides the base case we vary one factor at a time: network layout (clustered, mixed); shelf life $m \in \{2, 4\}$; lost-sales penalty $p_u \in \{10, 40\}$; fleet ratio $\rho \in \{1.10, 1.60\}$; routing cost $c^{\mathrm{r}} = 3$ per distance unit, which makes consolidation of deliveries more valuable; demand variability (negative-binomial size 3 instead of 8, i.e. coefficient of variation about 0.6 on regular days); partial LIFO issuing, where 30% of customers take the freshest units; and the real-data case. In the LIFO configuration all policies are tuned and trained under FIFO and evaluated under mixed issuing, which tests robustness to a misspecified issuing assumption that practitioners often face.

The configurations are not independent replications. All synthetic configurations except the high-variability one use the same training, validation and test demand paths (common random numbers across configurations). The inventory rules, with or without learning, set their orders without using the network geometry; the routing cost enters only their tuning and model selection. BSP and BSP-EW are tuned to the same parameters in all three layouts and therefore place exactly the same orders, differing only in travel cost. $\mathrm{UL}_\lambda$ is tuned to a lower visit trigger in the base layout ($\beta = 0.85$) than in the clustered and mixed layouts ($\beta = 0.9$), where skipping a store saves less travel; in the latter two it is tuned identically, and $\mathrm{UL}_\lambda$ and UL-DFL, whose training loss excludes routing, place identical orders in both. For the rule-based policies the clustered and mixed layouts therefore repeat one inventory comparison at two routing costs; they are informative mainly for the integrated policies, whose plans depend on the geometry.

### 5.5 Tuning, training and inference

All parameters are selected on validation episodes by closed-loop simulation: BSP and BSP-EW over $R \in \{1, 2\}$ and $\alpha \in \{0.8, 0.9, 0.95, 0.975, 0.99\}$; $\mathrm{UL}_\lambda$ over $\lambda = 1$ with $\alpha \in \{0.85, 0.875, 0.9, 0.925, 0.95, 0.96, 0.97, 0.975, 0.98, 0.99\}$ and $\beta \in \{0.5, 0.6, 0.7, 0.8, 0.85, 0.9, 0.95, 0.99\}$ and over $\lambda \in \{2, 3\}$ with $\alpha \in \{0.9, 0.95, 0.975, 0.99\}$ and $\beta \in \{0.7, 0.9, 0.95\}$; LA-Quantile over $\alpha \in \{0.7, 0.8, 0.9, 0.95, 0.975\}$; LA-Quantile2 over $\alpha_0 \in \{0.9, 0.95\}$ and $\alpha_1 \in \{0.6, 0.7, 0.8\}$; LA-Robust over $\Gamma \in \{0.25, 0.5, 1, 1.5\}$. A first, coarser grid for $\mathrm{UL}_\lambda$ ($\alpha \in \{0.9, 0.95, 0.975, 0.99\}$, $\beta \in \{0.7, 0.9, 0.95\}$) was refined because the rule tuned in hindsight on test episodes (Section 6.3) showed that it left part of the rule's potential unused; $\mathrm{UL}_\lambda$ is selected on the fine grid using validation episodes only, and UL-DFL is trained from the rule selected there. Likewise, a first implementation of ScenPO-SS drew its scenarios from bands of the full, unequally spaced quantile grid; it was replaced by the scenario set of the original method (Section 4.4). Appendix B reports both first versions.

UL-DFL is trained on all 200 training episodes for 30 epochs with Adam (learning rate $10^{-3}$, minibatches of ten episodes, gradient norm clipped at 5, $L_2$ penalty $10^{-4}$ on the output layer), from three random seeds in every configuration; NN-DFL and NB-DFL are trained identically but for 60 epochs. LA-DFL is trained on the decision states visited by the tuned LA-Quantile policy in 100 training episodes (2,100 states), with eight epochs, visit patterns re-optimised every second epoch, Adam (learning rate $2 \cdot 10^{-3}$, minibatch 64), a two-layer network with 64 SiLU units, and three seeds in the base case and one elsewhere. For all learners the network and the seed with the lowest closed-loop validation cost are kept. NN-DFL and NB-DFL were added after the primary hypotheses had been fixed; their comparisons are reported as secondary evidence.

The test episodes of the synthetic configurations had been evaluated in an earlier round of this study, which compared the rules and the lookahead policies before UL-DFL was designed, and the first 40 base-case test episodes were also used while UL-DFL was prototyped. To test the primary hypotheses on data that played no role in the design, tuning or training of any policy, the base case is evaluated on 100 new test episodes, drawn from the same demand model with new random seeds and evaluated once, after all policies had been tuned and trained. The other synthetic configurations are evaluated on the earlier test episodes and are reported as secondary evidence. The test episodes of the real-data case were used once.

We prespecify two families of primary hypotheses in the base case. For UL-DFL they compare it with $\text{UL}_\lambda$, BSP-EW, ScenPO-SS, LA-DFL and LA-Quantile2; for LA-DFL with LA-Quantile, ScenPO-SS, LA-Robust, BSP-EW and $\text{UL}_\lambda$. The unit of analysis is the episode; for each comparison we report the mean paired cost difference, a 95% paired bootstrap interval (10,000 resamples), paired $t$ and Wilcoxon signed-rank tests, and Holm-adjusted $p$-values within each family [31]. Relative differences between two policies are means of the episode-level relative differences, in percent of the comparator's cost. All other comparisons and configurations are reported with paired intervals as secondary evidence.

### 5.6 Optimisation settings

Lookahead MILPs are solved by HiGHS [33] through SciPy with a relative gap of 0.5% and time limits of 20 seconds (deterministic) and 60 seconds (scenario), with integer vehicle assignments on the implemented day and relaxed assignments on later lookahead days (Section 3.2). The LA-DFL oracle uses the same model with a 1% gap. If a time limit were reached the best incumbent would be used and its proven gap recorded; Section 6.8 reports solver statistics. All experiments run on a 16-core laptop computer (Intel Core Ultra 9 285H) with 12 to 15 parallel worker processes; the code, seeds and configuration files are in the replication package.

## 6 Results

### 6.1 Base case

Table 3 reports averages over the 100 fresh test episodes of the base case (Section 5) and Figure 1 decomposes the costs. Tuning selected order-up-to levels at the 0.95 quantile with daily delivery for BSP and BSP-EW, $\text{UL}_\lambda$ with $\lambda = 1$, $\alpha = 0.96$, $\beta = 0.85$, quantile level $\alpha^\star = 0.9$ for LA-Quantile, levels $(0.9, 0.8)$ for LA-Quantile2 and budget $\Gamma = 0.5$ for LA-Robust.

Table 3: Base case, fresh test episodes: mean cost per 21-day episode and its components, fill rate, waste rate (expired units per delivered unit) and store visits per day (100 episodes).

| Policy | Total cost (SD) | Routing | Holding | Waste | Lost sales | Fill rate (%) | Waste rate (%) | Visits/day |
|---|---|---|---|---|---|---|---|---|
| BSP | 10,846 (2,122) | 4,413 | 2,913 | 429 | 3,090 | 96.2 | 2.27 | 9.6 |
| BSP-EW | 10,868 (2,106) | 4,427 | 2,959 | 433 | 3,050 | 96.3 | 2.28 | 9.6 |
| $\text{UL}_\lambda$ | 10,740 (2,087) | 4,281 | 3,062 | 500 | 2,897 | 96.5 | 2.62 | 8.8 |
| UL-DFL | 10,632 (2,114) | 4,290 | 2,967 | 362 | 3,013 | 96.3 | 1.89 | 9.0 |
| NN-DFL | 10,834 (2,210) | 4,399 | 2,979 | 320 | 3,135 | 96.1 | 1.68 | 9.6 |
| NB-DFL | 10,813 (2,194) | 4,409 | 2,921 | 327 | 3,156 | 96.1 | 1.72 | 9.7 |
| LA-Mean | 19,098 (2,852) | 3,973 | 829 | 21 | 14,274 | 81.7 | 0.13 | 9.7 |
| LA-Quantile | 12,066 (2,259) | 3,519 | 4,004 | 1,081 | 3,463 | 95.7 | 5.27 | 7.8 |
| LA-Quantile2 | 11,385 (2,314) | 3,740 | 3,085 | 529 | 4,031 | 95.0 | 2.70 | 8.6 |
| LA-Robust | 11,826 (2,273) | 3,824 | 2,317 | 245 | 5,441 | 93.2 | 1.30 | 9.0 |
| ScenPO-SS | 11,831 (2,041) | 3,706 | 2,134 | 161 | 5,831 | 92.6 | 0.86 | 8.8 |
| LA-DFL | 11,452 (2,435) | 3,741 | 3,251 | 547 | 3,913 | 95.2 | 2.74 | 8.6 |
| PI | 6,002 (2,380) | 4,110 | 139 | 0 | 1,753 | 98.0 | 0.00 | 9.8 |

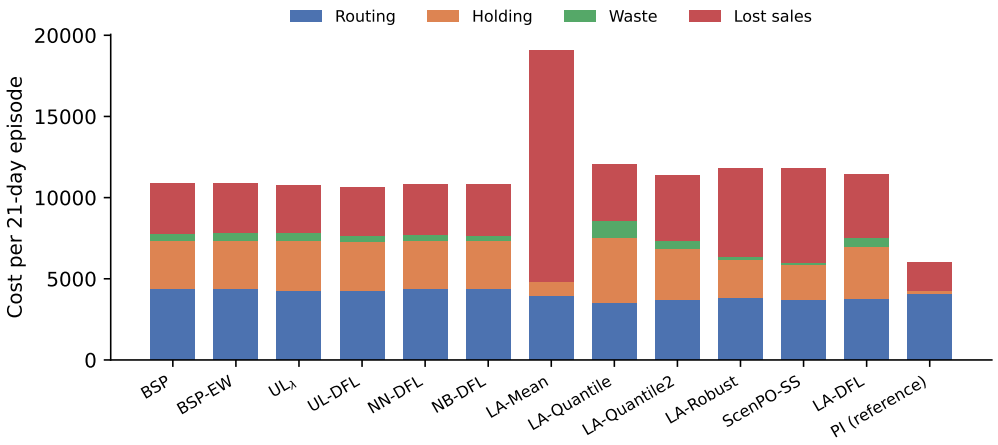



Figure 1: Cost composition of each policy in the base case (fresh test episodes).

Four observations stand out. First, uncertainty is expensive: with perfect demand information the same lookahead costs 6,002, a little more than half of the best implementable policy, almost entirely because lost sales and holding vanish. Second, point forecasts are a poor input for perishable replenishment. LA-Mean plans to sell exactly the expected demand and loses 14,274 per episode in lost sales (fill rate 81.7%), as Proposition 2 suggests when $p_u$ is four times $p_s$. Third, among the policies without learning, the three stock-age-based inventory rules are the cheapest: UL$_\lambda$, BSP and BSP-EW cost between 10,740 and 10,868, with fill rates of 96.2–96.5% and waste rates of 2.3–2.6%. With daily review, little of the current stock is at risk of expiring before it is sold, so the waste correction of BSP-EW rarely changes an order.

Fourth, integration with routing does reduce travel but loses more on the inventory side. The uncertainty-aware lookahead policies visit 7.8–9.0 stores per day instead of 8.8–9.6 and save about 460–910 in routing, but they miss the balance between waste and service that the rules strike, in one of two directions. LA-Quantile holds 31–37% more stock than the rules and wastes twice as much (waste rate 5.27%): a deterministic plan that covers two or three days with quantile targets stacks a safety buffer on every covered day, even though the next day's re-planning will correct any shortfall. LA-Quantile2, which keeps the 0.9 quantile only for the implemented day and plans later days at the 0.8 quantile, halves the waste rate of LA-Quantile (2.70% vs. 5.27%) and costs 5.6% less. LA-Robust and ScenPO-SS err in the other direction: they waste little (1.30% and 0.86%) but serve only 93.2% and 92.6% of demand. The scenario set of ScenPO-SS contains no demand above the predicted 0.9 quantile of any store-day, so its single plan protects against less demand than the asymmetry between lost sales and waste warrants.

A longer lookahead does not change this ranking. With horizons of $H = 4$ and $H = 5$ days, LA-Quantile2 re-tuned on the validation episodes (levels $(0.9, 0.7)$ for both) is 1.2% and 1.1% cheaper on the fresh test episodes than with $H = 3$ (95% CIs $[-218, -50]$ and $[-225, -45]$), but it remains 5.7% and 5.8% more expensive than UL-DFL and 4.6% and 4.7% more expensive than UL$_\lambda$. The gain from looking further ahead levels off after four days, at 1.9 and 2.2 seconds per decision instead of 0.90.

Table 4: Paired episode-level comparisons of UL-DFL with every other policy (base case, fresh test episodes, 100 episodes). Negative differences favour UL-DFL. Rel.: mean of the episode-level relative differences, in percent of the comparator's cost. Holm adjustment over the five prespecified primary comparisons of UL-DFL.

| Comparison | Mean diff. | 95% bootstrap CI | Rel. (%) | Wins (%) | $p$ ($t$) | $p$ (Wilcoxon) | Holm $p$ |
|---|---|---|---|---|---|---|---|
| UL-DFL − BSP | -213.8 | [-265.8, -160.5] | -2.0 | 80 | <0.001 | <0.001 | – |
| UL-DFL − BSP-EW | -236.1 | [-289.0, -182.5] | -2.2 | 80 | <0.001 | <0.001 | <0.001 |
| UL-DFL − UL$_\lambda$ | -107.4 | [-153.1, -60.5] | -1.1 | 67 | <0.001 | <0.001 | <0.001 |
| UL-DFL − NN-DFL | -201.3 | [-254.4, -149.7] | -1.7 | 78 | <0.001 | <0.001 | – |
| UL-DFL − NB-DFL | -180.5 | [-225.4, -136.3] | -1.6 | 80 | <0.001 | <0.001 | – |
| UL-DFL − LA-Mean | -8,465.5 | [-8,764.1, -8,175.6] | -44.4 | 100 | <0.001 | <0.001 | – |
| UL-DFL − LA-Quantile | -1,434.2 | [-1,585.6, -1,278.2] | -11.8 | 98 | <0.001 | <0.001 | – |
| UL-DFL − LA-Quantile2 | -752.9 | [-872.7, -629.9] | -6.3 | 92 | <0.001 | <0.001 | <0.001 |
| UL-DFL − LA-Robust | -1,193.9 | [-1,313.2, -1,076.2] | -10.0 | 99 | <0.001 | <0.001 | – |
| UL-DFL − ScenPO-SS | -1,199.2 | [-1,342.1, -1,056.2] | -10.3 | 95 | <0.001 | <0.001 | <0.001 |
| UL-DFL − LA-DFL | -819.9 | [-958.9, -684.2] | -6.7 | 95 | <0.001 | <0.001 | <0.001 |
| UL-DFL − PI | 4,630.0 | [4,454.8, 4,800.8] | 87.0 | 0 | <0.001 | <0.001 | – |

UL-DFL is the cheapest implementable policy (Table 4). It costs 1.1% less than the re-tuned UL$_\lambda$ from which it starts (95% CI [-153, -60], lower in 67% of the episodes), 2.2% less than BSP-EW, 10.3% less than ScenPO-SS, 6.3% less than LA-Quantile2 and 6.7% less than LA-DFL; all five primary comparisons are significant after Holm adjustment ($p < 0.001$). The gain over UL$_\lambda$ is a waste reduction: UL-DFL lowers the waste rate from 2.62% to 1.89%, i.e. it throws away 28% fewer units, and carries 3% less stock, while its fill rate falls slightly (from 96.5% to 96.3%) and its routing cost is practically unchanged. On the earlier test episodes of the base case the comparison is the same (0.8% below UL$_\lambda$, CI [-130, -44]; Table 10).

Table 5: Paired episode-level comparisons of LA-DFL with every other policy (base case, fresh test episodes, 100 episodes). Negative differences favour LA-DFL. Rel.: as in Table 4. Holm adjustment over the five prespecified primary comparisons of LA-DFL.

| Comparison | Mean diff. | 95% bootstrap CI | Rel. (%) | Wins (%) | $p$ ($t$) | $p$ (Wilcoxon) | Holm $p$ |
|---|---|---|---|---|---|---|---|
| LA-DFL − BSP | 606.1 | [461.9, 749.6] | 5.4 | 20 | <0.001 | <0.001 | – |
| LA-DFL − BSP-EW | 583.8 | [437.8, 729.3] | 5.2 | 20 | <0.001 | <0.001 | <0.001 |
| LA-DFL − UL$_\lambda$ | 712.5 | [567.6, 859.9] | 6.4 | 16 | <0.001 | <0.001 | <0.001 |
| LA-DFL − UL-DFL | 819.9 | [684.2, 958.9] | 7.6 | 5 | <0.001 | <0.001 | – |
| LA-DFL − NN-DFL | 618.6 | [485.5, 751.2] | 5.7 | 17 | <0.001 | <0.001 | – |
| LA-DFL − NB-DFL | 639.4 | [495.3, 782.4] | 5.9 | 16 | <0.001 | <0.001 | – |
| LA-DFL − LA-Mean | -7,645.6 | [-7,976.6, -7,318.0] | -40.2 | 100 | <0.001 | <0.001 | – |
| LA-DFL − LA-Quantile | -614.4 | [-734.6, -491.0] | -5.3 | 84 | <0.001 | <0.001 | <0.001 |
| LA-DFL − LA-Quantile2 | 66.9 | [-23.1, 156.4] | 0.5 | 46 | 0.148 | 0.296 | – |
| LA-DFL − LA-Robust | -374.0 | [-513.3, -231.5] | -3.3 | 71 | <0.001 | <0.001 | <0.001 |
| LA-DFL − ScenPO-SS | -379.3 | [-604.4, -155.2] | -3.4 | 66 | 0.001 | 0.001 | 0.001 |
| LA-DFL − PI | 5,449.9 | [5,243.4, 5,659.1] | 100.8 | 0 | <0.001 | <0.001 | – |

LA-DFL improves the deterministic lookahead from which it starts (Table 5). It reduces cost by 5.3% relative to LA-Quantile (95% CI [-735, -491], lower in 84% of the episodes), by 3.3% relative to LA-Robust and by 3.4% relative to ScenPO-SS (Holm-adjusted $p = 0.001$). It does so by halving the waste rate of LA-Quantile (2.74% vs. 5.27%) while accepting somewhat more lost sales. The simple LA-Quantile2, however, is 0.5% cheaper still (not significant, CI of LA-DFL minus LA-Quantile2 [-23, 156]): the gain of LA-DFL over LA-Quantile is captured by a day-specific quantile level, which LA-DFL discovered (Section 6.2) but did not improve on. LA-DFL also remains 5.2% more expensive than BSP-EW and 6.4% more expensive than UL$_\lambda$, both significant.

### 6.2 What the decision-focused corrections learn

Table 6 summarises the learned corrections of UL-DFL on the delivery decisions it takes in the 100 earlier test episodes of the base case (the rule triggers a delivery on 90% of the store-days; on the others the correction has no effect). On average the network lowers the order-up-to level slightly, from the tuned 0.96 quantile to an implied service level of 0.94. More important is *when* it deviates, along two dimensions that a one-day order-up-to rule cannot see. On promotion days it lowers the level to an implied service level of about 0.89, i.e. by about a third of a day of mean demand. Promotions raise the forecast spread, so a high quantile lies far above the expected demand, and whatever is left when the promotion ends meets regular demand and ages. Over the week it raises the level on Thursday (implied level 0.97), keeps it near the tuned level on Wednesday and Friday, and lowers it from Saturday to Monday (0.91–0.92): a unit left over before the weekend is sold in the Saturday peak, whereas a unit left over after it meets lower demand for the rest of its shelf life. The age of the stock on hand matters little, because the rule's order quantity already nets out the stock projected to expire; the level is only slightly lower when units expire on the day (0.933 vs. 0.942). Both main corrections reduce the stock that is still on the shelf when demand falls, which is where the waste of the rule arises.

Table 6: Learned corrections of UL-DFL on the delivery decisions of 100 earlier test episodes (base case): mean correction $\delta_\theta$ of the order-up-to level in units of the forecast scale, implied service level $\Phi(z_{0.96} + \delta_\theta)$, and change of the order-up-to level in days of mean store demand. Stock is measured in days of mean store demand.

| Delivery decisions | $n$ | Correction $\delta_\theta$ (SD) | Implied service level | Change of $S$ (days) |
|---|---|---|---|---|
| all delivery decisions | 18,952 | -0.13 (0.30) | 0.940 | -0.07 |
| no promotion | 16,404 | -0.08 (0.27) | 0.947 | -0.03 |
| promotion | 2,548 | -0.48 (0.27) | 0.890 | -0.37 |
| Mon | 2,715 | -0.31 (0.20) | 0.921 | -0.14 |
| Tue | 2,653 | -0.14 (0.22) | 0.941 | -0.07 |
| Wed | 2,745 | +0.01 (0.20) | 0.958 | -0.01 |
| Thu | 2,725 | +0.21 (0.23) | 0.972 | +0.09 |
| Fri | 2,755 | +0.06 (0.23) | 0.961 | +0.02 |
| Sat | 2,854 | -0.39 (0.20) | 0.908 | -0.23 |
| Sun | 2,505 | -0.35 (0.21) | 0.914 | -0.17 |
| stock <0.25 d | 4,450 | -0.09 (0.30) | 0.945 | -0.05 |
| stock 0.25-0.5 d | 2,595 | -0.12 (0.29) | 0.941 | -0.06 |
| stock >0.5 d | 11,907 | -0.15 (0.31) | 0.937 | -0.08 |
| no expiring units | 13,299 | -0.11 (0.30) | 0.942 | -0.06 |
| units expiring today | 5,653 | -0.19 (0.30) | 0.933 | -0.10 |

Table 7 reports the corrections of LA-DFL on the decision states of 20 earlier test episodes. Here the dominant pattern is temporal within the plan. For the implemented day the network keeps the target at about the 0.91 quantile, essentially the tuned LA-Quantile level, but it lowers the targets of the second and third lookahead days to about the 0.80 and 0.77 quantiles. In inventory terms, the network has learned that only today's shortfall is irreversible: tomorrow's planned demand will be re-planned with tomorrow's stock and forecast, so a full safety buffer on that day mostly turns into carried stock and, with a three-day shelf life, into waste. This is precisely the recourse that a deterministic lookahead ignores and that Proposition 3 lets the gradient account for. Context-specific corrections are small: targets are lowered somewhat more on promotion days and slightly less when the store holds more or older stock. The day-specific levels learned by LA-DFL are those that the tuning of LA-Quantile2 selects, $(0.9, 0.8)$, which explains why the two policies perform alike.

Table 7: Learned corrections of LA-DFL on the decision states of 20 earlier test episodes (base case): mean correction $\delta_\theta$ in forecast-scale units, implied quantile level of the planning target, and target relative to the mean forecast. $k$ is the lookahead offset; stock is measured in days of mean store demand.

| Targets | $n$ | Correction $\delta_\theta$ (SD) | Implied quantile level | Target / mean forecast |
|---|---|---|---|---|
| all | 12,600 | -0.37 (0.38) | 0.826 | 1.46 |
| k=0 | 4,200 | +0.12 (0.15) | 0.910 | 1.67 |
| k=1 | 4,200 | -0.53 (0.15) | 0.804 | 1.38 |
| k=2 | 4,200 | -0.69 (0.17) | 0.765 | 1.31 |
| no promotion | 11,078 | -0.36 (0.39) | 0.828 | 1.45 |
| promotion | 1,522 | -0.47 (0.32) | 0.818 | 1.53 |
| stock <0.25 d | 2,694 | -0.42 (0.39) | 0.816 | 1.43 |
| stock 0.25-0.75 d | 4,059 | -0.40 (0.38) | 0.821 | 1.45 |
| stock >0.75 d | 5,847 | -0.33 (0.38) | 0.835 | 1.48 |
| no expiring units | 8,088 | -0.39 (0.38) | 0.822 | 1.45 |
| units expiring today | 4,512 | -0.33 (0.38) | 0.834 | 1.48 |

For both learners the closed-loop validation cost improved during the first three to six epochs and deteriorated afterwards (Figure 2), which is why model selection on closed-loop validation cost is part of the method. The result does not depend on a lucky seed (Table 8): on the fresh base-case test episodes the three UL-DFL networks are 0.98–1.05% cheaper than $\mathrm{UL}_\lambda$, and on the earlier test episodes the three LA-DFL networks are 3.6–4.9% cheaper than LA-Quantile.

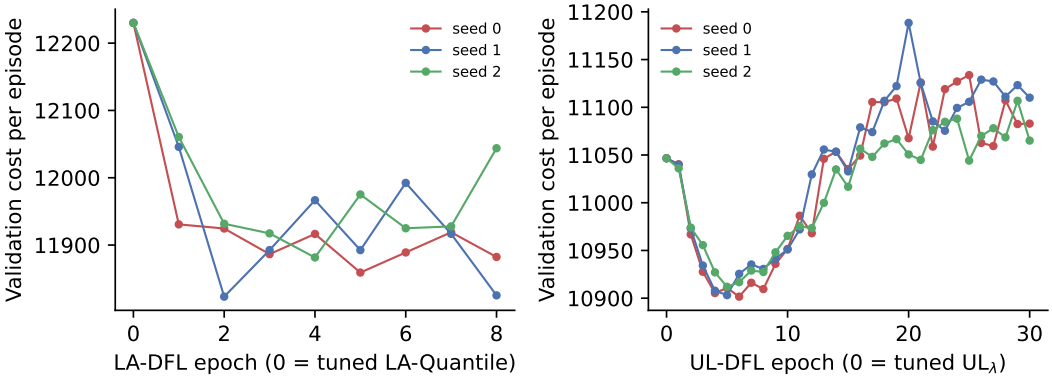



Figure 2: Closed-loop validation cost by training epoch and seed (base case). Left: LA-DFL, epoch 0 is the tuned LA-Quantile policy. Right: UL-DFL, epoch 0 is the tuned $\mathrm{UL}_\lambda$ rule.

Table 8: Test-cost difference (%, mean of the episode-level relative differences) of each trained network relative to the policy it starts from, by seed; "selected" is the network chosen on validation cost. "Base (fresh test)": fresh test episodes; all other rows: earlier test episodes.

| Setting | Comparison | Selected (%) | Seed 0 | Seed 1 | Seed 2 |
|---|---|---|---|---|---|
| Base (fresh test) | UL-DFL vs $UL_\lambda$ | -1.05 | -1.05 | -1.02 | -0.98 |
| Base (fresh test) | NN-DFL vs $UL_\lambda$ | +0.75 | +0.49 | +0.86 | +0.75 |
| Base (fresh test) | NB-DFL vs $UL_\lambda$ | +0.57 | +0.52 | +0.57 | +0.89 |
| Base | UL-DFL vs $UL_\lambda$ | -0.81 | -0.81 | -0.75 | -0.93 |
| Base | NN-DFL vs $UL_\lambda$ | +1.25 | +1.26 | +1.18 | +1.25 |
| Base | NB-DFL vs $UL_\lambda$ | +1.00 | +0.97 | +1.00 | +1.26 |
| Base | LA-DFL vs LA-Quantile | -4.86 | -3.71 | -4.86 | -3.59 |
| Clustered | UL-DFL vs $UL_\lambda$ | -1.21 | -1.21 | -1.17 | -1.15 |
| Clustered | NN-DFL vs $UL_\lambda$ | +0.02 | +0.08 | +0.02 | +0.16 |
| Clustered | NB-DFL vs $UL_\lambda$ | -0.27 | -0.21 | -0.27 | +0.18 |
| Mixed | UL-DFL vs $UL_\lambda$ | -1.02 | -1.02 | -1.02 | -0.95 |
| Mixed | NN-DFL vs $UL_\lambda$ | +0.28 | +0.25 | +0.04 | +0.28 |
| Mixed | NB-DFL vs $UL_\lambda$ | -0.03 | +0.01 | -0.03 | +0.29 |
| $m = 2$ | UL-DFL vs $UL_\lambda$ | -1.12 | -1.11 | -1.00 | -1.12 |
| $m = 2$ | NN-DFL vs $UL_\lambda$ | -0.49 | -0.61 | -0.55 | -0.49 |
| $m = 2$ | NB-DFL vs $UL_\lambda$ | -0.42 | -0.34 | -0.21 | -0.42 |
| $m = 4$ | UL-DFL vs $UL_\lambda$ | -0.30 | -0.30 | -0.27 | -0.19 |
| $m = 4$ | NN-DFL vs $UL_\lambda$ | +2.49 | +2.40 | +2.47 | +2.42 |
| $m = 4$ | NB-DFL vs $UL_\lambda$ | +1.66 | +1.66 | +2.09 | +1.96 |
| $p_u = 10$ | UL-DFL vs $UL_\lambda$ | -0.96 | -0.92 | -0.96 | -0.89 |
| $p_u = 10$ | NN-DFL vs $UL_\lambda$ | +1.15 | +1.25 | +1.25 | +1.15 |
| $p_u = 10$ | NB-DFL vs $UL_\lambda$ | +0.84 | +0.88 | +0.84 | +0.90 |
| $p_u = 40$ | UL-DFL vs $UL_\lambda$ | -0.84 | -0.89 | -0.84 | -0.88 |
| $p_u = 40$ | NN-DFL vs $UL_\lambda$ | +0.68 | +0.68 | +0.63 | +1.08 |
| $p_u = 40$ | NB-DFL vs $UL_\lambda$ | +1.30 | +1.59 | +1.30 | +1.55 |
| $\rho = 1.10$ | UL-DFL vs $UL_\lambda$ | -0.81 | -0.81 | -0.76 | -0.76 |
| $\rho = 1.10$ | NN-DFL vs $UL_\lambda$ | +0.74 | +0.47 | +0.74 | +0.79 |
| $\rho = 1.10$ | NB-DFL vs $UL_\lambda$ | +0.65 | +0.66 | +0.65 | +0.76 |
| $\rho = 1.60$ | UL-DFL vs $UL_\lambda$ | -0.30 | -0.31 | -0.30 | -0.35 |
| $\rho = 1.60$ | NN-DFL vs $UL_\lambda$ | +1.54 | +1.66 | +1.75 | +1.54 |
| $\rho = 1.60$ | NB-DFL vs $UL_\lambda$ | +1.32 | +1.33 | +1.32 | +1.30 |
| High var. | UL-DFL vs $UL_\lambda$ | -0.00 | +0.04 | -0.00 | -0.18 |
| High var. | NN-DFL vs $UL_\lambda$ | +0.86 | +0.74 | +0.92 | +0.86 |
| High var. | NB-DFL vs $UL_\lambda$ | +1.30 | +1.39 | +1.30 | +1.56 |
| $c^r = 3$ | UL-DFL vs $UL_\lambda$ | -0.81 | -0.93 | -0.81 | -0.73 |
| $c^r = 3$ | NN-DFL vs $UL_\lambda$ | +4.56 | +4.69 | +4.65 | +4.56 |
| $c^r = 3$ | NB-DFL vs $UL_\lambda$ | +4.50 | +4.30 | +4.50 | +4.39 |
| 30% LIFO | UL-DFL vs $UL_\lambda$ | -0.80 | -0.80 | -0.75 | -0.82 |
| 30% LIFO | NN-DFL vs $UL_\lambda$ | +1.75 | +1.67 | +2.02 | +1.75 |
| 30% LIFO | NB-DFL vs $UL_\lambda$ | +1.43 | +1.39 | +1.43 | +1.56 |
| 30% LIFO | LA-DFL vs LA-Quantile | -4.94 | -4.43 | -4.94 | -3.43 |
| Real data | UL-DFL vs $UL_\lambda$ | +0.62 | +0.51 | +0.09 | +0.62 |
| Real data | NN-DFL vs $UL_\lambda$ | +1.44 | +1.78 | +1.44 | +1.60 |
| Real data | NB-DFL vs $UL_\lambda$ | +1.13 | +1.13 | +1.37 | +1.70 |

### 6.3 Can re-tuning the rule reproduce UL-DFL?

A natural objection is that UL-DFL merely finds better values of the rule's parameters. For the first version of this study the objection was partly right: $UL_\lambda$ had been tuned on a coarse grid, and about half of the advantage of UL-DFL over it disappeared once the rule was tuned on a fine grid (Appendix B). We therefore tune the rule on the fine grid and train UL-DFL from it. Figure 3 places UL-DFL in the waste–service plane together with 124 parameterisations of the rules, all evaluated on the earlier base-case test episodes: $UL_\lambda$ with $\lambda = 1$, ten service levels $\alpha$ between 0.85 and 0.99 and eight trigger levels $\beta$ between 0.5 and 0.99, $UL_\lambda$ with $\lambda \in \{2, 3\}$, and daily BSP and BSP-EW at the same ten service levels. No parameterisation serves at least

UL-DFL's fill rate (96.3%) with no more waste: at that fill rate the least waste any rule achieves is 2.49% of the delivered units, against 2.09% for UL-DFL. The learned corrections move the policy beyond the waste–service frontier of the rules, which no choice of a single service level and trigger can do.

The comparison in total cost favours the rules, because the cheapest of the 124 rules is selected on the test episodes themselves, an advantage UL-DFL does not have. This rule, $\mathrm{UL}_\lambda$ with $\alpha = 0.96$ and $\beta = 0.8$, costs 10,715 per episode; UL-DFL is still 0.6% cheaper (95% CI $[-113, -17]$) and wastes 25% less (2.09% vs. 2.80% of the delivered units). On the fresh test episodes, on which all 124 rules were run in the same way, the result is the same: no parameterisation serves at least UL-DFL's fill rate (96.3%) with no more waste (the least waste at that fill rate is 2.60% of the delivered units, against 1.89% for UL-DFL), and UL-DFL is 1.0% cheaper than the best rule in hindsight, again $\alpha = 0.96$ and $\beta = 0.8$ (95% CI $[-150, -46]$). The real-data case gives a different picture: there the rule selected on validation episodes ($\alpha = 0.96$, $\beta = 0.85$) is also the cheapest rule in hindsight, and five parameterisations serve at least UL-DFL's fill rate with no more waste. On these data the learned corrections do not move the policy beyond the frontier of the rules (Section 6.6).

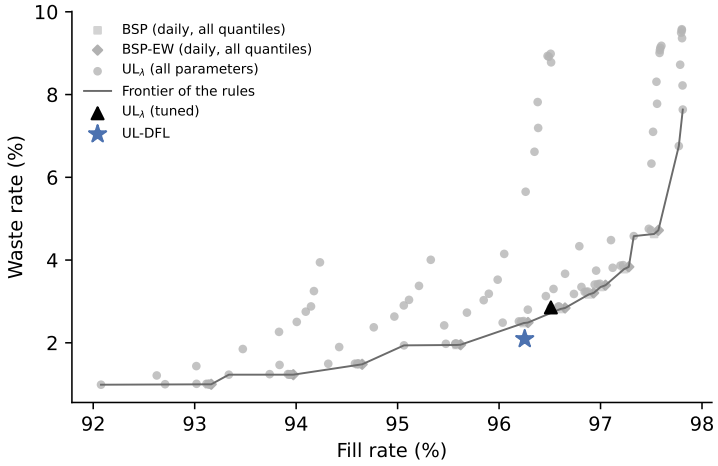



Figure 3: Waste rate against fill rate of 124 parameterisations of the rules and of UL-DFL (base case, 100 earlier test episodes). The line connects the rule parameterisations that are not dominated in both measures.

### 6.4 Does the learner need the rule?

The converse question is whether the rule contributes anything once a network is trained on realised cost. NN-DFL answers it: it has the same inputs, differentiable simulation, loss, seeds and model selection as UL-DFL, but outputs the delivery directly, without the visit trigger and order-up-to structure of $\mathrm{UL}_\lambda$. On the fresh base-case test episodes it costs 10,834 per episode, 1.7% more than UL-DFL (95% CI of UL-DFL minus NN-DFL [-254, -150]) and more than the tuned $\mathrm{UL}_\lambda$ it would replace. It does learn to waste little (waste rate 1.68%), but it loses more sales and, lacking a trigger, visits more stores (9.6 per day against 9.0). Across the thirteen configurations UL-DFL is significantly cheaper than NN-DFL in twelve, by 0.6–5.1% (Table 10), and not significantly different on the real data. The gap is largest with expensive routing, where the trigger of UL-DFL cuts store visits to 6.1 per day while NN-DFL keeps visiting 9.6. NN-DFL is never significantly cheaper than $\mathrm{UL}_\lambda$ itself and significantly more expensive in nine configurations.

NN-DFL lacks two things at once: the order-up-to structure and the rule it starts from. NB-DFL separates them. It is the same learner with the structure that neural inventory policies

commonly use: the network outputs an order-up-to level, the store receives that level minus its usable stock, and training starts from a level of one day of mean demand, without the forecast quantile and the visit trigger of $UL_\lambda$. The structure adds almost nothing. On the fresh base-case test episodes NB-DFL costs 10,813 per episode, 1.6% more than UL-DFL (95% CI of UL-DFL minus NB-DFL [-225, -136]) and about as much as NN-DFL. Across the configurations UL-DFL is significantly cheaper than NB-DFL in twelve, by 0.7–5.0%, and not significantly different on the real data. NB-DFL differs significantly from NN-DFL only in two configurations, and it is never significantly cheaper than $UL_\lambda$ and significantly more expensive in ten. Both networks reach their lowest validation cost by epoch 33 of 60, mostly within the first 15, so more training would not close the gap. With expensive routing NB-DFL visits 9.5 stores per day, against 6.1 for UL-DFL. These comparisons are secondary, because both ablations were added after the primary hypotheses were fixed, but they are consistent: with the same network and learning signal, the gain comes from starting at the tuned rule and keeping its visit trigger, not from the order-up-to structure alone.

### 6.5 Sensitivity analysis

Table 9 reports test costs across the configurations, and Tables 10 and 11 the paired differences of the two learned policies. Five patterns emerge across the thirteen configurations, which are evaluated on the earlier test episodes (Section 5). (i) UL-DFL is significantly cheaper than the re-tuned $UL_\lambda$ in nine configurations, by 0.8–1.2% (in eight of twelve if the clustered and mixed layouts, in which both policies place identical orders, are counted once). All three training seeds are cheaper than the rule in every configuration except high demand variability, where their signs differ, and the real data, where all three are more expensive (Table 8). UL-DFL is the cheapest implementable policy in ten configurations; the rule itself is cheapest with high variability and on the real data, and LA-Quantile2 with expensive routing, where UL-DFL is level with it (+0.1%, 95% CI [-126, 189]). In twelve configurations UL-DFL wastes 7–30% fewer units than the rule, at a fill rate within 0.6 points. The configurations without a significant gain are instructive. With a four-day shelf life the rule wastes only 0.66% of the delivered units, so the corrections have little to remove: UL-DFL lowers the waste rate to 0.46% but saves only 0.3% in total (CI [-66, 10]). With a loose fleet the saving is 0.3% (CI [-62, 5]), and with high demand variability the waste saving (18% fewer units) is offset by lost sales (fill rate 92.8% instead of 93.4%). With expensive routing the gain has a different source: UL-DFL raises the order-up-to levels so that stores are visited less often (6.1 instead of 7.0 visits per day) and accepts 39% more waste. On the real data UL-DFL is 0.6% more expensive than the rule (CI [12, 163]; Section 6.6). The value of learning the order-up-to level is therefore largest when perishability makes the rule's single service level costly but demand is predictable enough from context to act on it, and it requires corrections that carry over from the training data to the episodes on which the policy is used. (ii) Without learning, the stock-age-based rules are cheaper than every integrated policy in twelve configurations: the best rule is 2.0–11.9% below the best integrated policy. The advantage is largest when lost sales are expensive ($p_u = 40$, 11.9%) and in the clustered layout (9.3%), where all stores are close to each other, so that skipping a store saves little travel. It is smallest with a low lost-sales penalty ($p_u = 10$, 2.0%), where the high service level of the rules is worth less, and with a two-day shelf life or a loose or tight fleet (3.2–3.6%). Partial LIFO issuing raises the waste rate of every policy, by 41–105%, but does not change which policies are best. (iii) With three times higher routing cost, integration pays: LA-Quantile2 is the cheapest policy, 1.1% below $UL_\lambda$ and level with UL-DFL, and daily base-stock (BSP-EW) is more expensive than every uncertainty-aware lookahead. The trigger of $UL_\lambda$, which skips stores whose stock suffices, keeps the deliver-up-to-level rules competitive. (iv) Among the integrated policies, LA-Quantile2 is best in ten configurations and ScenPO-SS in three ($m = 2$, $p_u = 40$ and the tight fleet). LA-Mean is the most expensive policy in all thirteen. ScenPO-SS, whose scenario set does not extend beyond the predicted 0.9 quantile, under-stocks in every configuration (fill rate 87.2–94.7%). Sampling its

scenarios from the full quantile grid instead lowers its cost in twelve configurations (significantly in nine) and would make it the cheapest policy with expensive routing, but would not change the cheapest policy elsewhere (Appendix B). (v) LA-DFL is significantly cheaper than LA-Quantile in seven configurations (by 1.7–5.0%), not significantly different in five and more expensive with a two-day shelf life, and it is more expensive than LA-Quantile2 in every configuration (by 0.3–5.7%, significant in nine): on top of the lookahead, the value of decision-focused learning lies in revealing that the safety stock should depend on the planning day, which a two-parameter policy then implements.

Table 9: Mean test cost per episode by configuration (earlier test episodes; one factor changed at a time; bold: lowest implementable policy), and fill rates of selected policies.

| Policy | Base | Clustered | Mixed | $m = 2$ | $m = 4$ | $p_u = 10$ | $p_u = 40$ | $\rho = 1.10$ | $\rho = 1.60$ | High var. | $c^r = 3$ | 30% LIFO | Real data |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BSP | 10,851 | 8,266 | 10,206 | 12,607 | 10,490 | 9,056 | 13,480 | 12,221 | 9,868 | 15,011 | 19,633 | 11,208 | 13,068 |
| BSP-EW | 10,885 | 8,289 | 10,235 | 12,633 | 10,490 | 9,082 | 13,496 | 12,227 | 9,922 | 15,089 | 19,704 | 11,256 | 13,090 |
| $UL_\lambda$ | 10,737 | 8,267 | 10,194 | 12,548 | 10,280 | 8,956 | 13,328 | 12,086 | 9,746 | **14,839** | 18,830 | 11,061 | **12,937** |
| UL-DFL | **10,650** | **8,169** | **10,092** | **12,411** | **10,253** | **8,867** | **13,201** | **12,007** | **9,716** | 14,850 | 18,663 | **10,974** | 13,023 |
| NN-DFL | 10,871 | 8,263 | 10,221 | 12,483 | 10,534 | 9,052 | 13,396 | 12,179 | 9,892 | 14,974 | 19,645 | 11,253 | 13,093 |
| NB-DFL | 10,843 | 8,245 | 10,190 | 12,489 | 10,451 | 9,025 | 13,475 | 12,169 | 9,871 | 15,028 | 19,628 | 11,220 | 13,087 |
| LA-Mean | 19,371 | 17,051 | 18,362 | 19,769 | 19,350 | 12,074 | 33,960 | 19,362 | 19,407 | 25,609 | 24,289 | 19,400 | 21,030 |
| LA-Quantile | 12,122 | 9,735 | 11,520 | 13,954 | 11,289 | 9,314 | 15,490 | 13,327 | 10,405 | 16,398 | 19,157 | 12,574 | 14,478 |
| LA-Quantile2 | 11,433 | 9,110 | 10,830 | 13,553 | 10,977 | 9,136 | 15,436 | 13,047 | 10,071 | 15,631 | **18,630** | 11,787 | 14,185 |
| LA-Robust | 11,948 | 9,545 | 11,209 | 13,622 | 11,221 | 9,173 | 15,295 | 13,291 | 10,184 | 15,851 | 18,983 | 12,110 | 14,560 |
| ScenPO-SS | 11,927 | 9,685 | 11,685 | 12,963 | 11,833 | 9,849 | 15,133 | 12,532 | 12,153 | 16,777 | 19,300 | 12,165 | 14,329 |
| LA-DFL | 11,548 | 9,294 | 11,089 | 14,295 | 11,015 | 9,258 | 15,498 | 13,260 | 10,146 | 16,101 | 19,058 | 11,974 | 14,333 |
| PI (reference) | 5,758 | 3,352 | 5,180 | 6,121 | 5,752 | 5,005 | 7,260 | 7,678 | 4,728 | 7,097 | 13,589 | 5,758 | 7,329 |
| Fill rate (%), UL-DFL | 96.3 | 96.2 | 96.2 | 94.9 | 96.6 | 94.6 | 97.0 | 94.7 | 97.0 | 92.8 | 95.4 | 96.2 | 93.6 |
| Fill rate (%), LA-DFL | 95.1 | 95.2 | 95.1 | 93.1 | 95.4 | 93.4 | 95.8 | 92.8 | 96.3 | 91.9 | 94.9 | 94.9 | 92.1 |
| Fill rate (%), ScenPO-SS | 92.4 | 92.7 | 92.0 | 92.1 | 92.4 | 88.0 | 94.7 | 91.9 | 91.5 | 87.2 | 91.1 | 92.3 | 89.2 |
| Fill rate (%), LA-Quantile | 95.7 | 96.2 | 96.1 | 91.2 | 95.9 | 92.7 | 95.8 | 91.6 | 96.3 | 89.1 | 93.0 | 95.6 | 92.8 |
| Fill rate (%), BSP-EW | 96.3 | 96.3 | 96.3 | 94.5 | 96.3 | 94.7 | 97.0 | 94.5 | 97.4 | 93.8 | 96.3 | 96.2 | 93.8 |

Table 10: Paired cost difference of UL-DFL relative to each comparator, in percent of the comparator's cost, by configuration (earlier test episodes). Bold: 95% paired bootstrap interval excludes zero.

| UL-DFL | Base | Clustered | Mixed | $m = 2$ | $m = 4$ | $p_u = 10$ | $p_u = 40$ | $\rho = 1.10$ | $\rho = 1.60$ | High var. | $c^r = 3$ | 30% LIFO | Real data |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| vs $UL_\lambda$ | **-0.8** | **-1.2** | **-1.0** | **-1.1** | -0.3 | **-1.0** | **-0.8** | **-0.8** | -0.3 | -0.0 | **-0.8** | **-0.8** | **+0.6** |
| vs BSP-EW | **-2.2** | **-1.5** | **-1.4** | **-1.8** | **-2.2** | **-2.4** | **-2.2** | **-1.9** | **-2.1** | **-1.8** | **-5.4** | **-2.6** | -0.5 |
| vs ScenPO-SS | **-10.8** | **-15.9** | **-13.6** | **-4.4** | **-13.4** | **-9.9** | **-13.2** | **-4.7** | **-19.6** | **-11.5** | **-3.4** | **-9.9** | **-8.3** |
| vs LA-DFL | **-7.5** | **-11.4** | **-8.5** | **-12.3** | **-6.7** | **-4.1** | **-14.6** | **-9.4** | **-4.1** | **-7.8** | **-2.1** | **-7.9** | **-9.6** |
| vs LA-Quantile2 | **-6.6** | **-9.8** | **-6.1** | **-7.6** | **-6.5** | **-2.9** | **-14.3** | **-7.8** | **-3.4** | **-5.0** | +0.1 | **-6.6** | **-8.4** |
| vs NN-DFL | **-2.0** | **-1.1** | **-1.2** | **-0.6** | **-2.7** | **-2.1** | **-1.4** | **-1.5** | **-1.8** | **-0.8** | **-5.1** | **-2.4** | -0.8 |
| vs NB-DFL | **-1.7** | **-0.9** | **-0.9** | **-0.7** | **-1.9** | **-1.8** | **-2.0** | **-1.4** | **-1.5** | **-1.2** | **-5.0** | **-2.1** | -0.5 |

Table 11: Paired cost difference of LA-DFL relative to each comparator, in percent of the comparator's cost, by configuration (earlier test episodes). Bold: 95% paired bootstrap interval excludes zero.

| LA-DFL | Base | Clustered | Mixed | $m = 2$ | $m = 4$ | $p_u = 10$ | $p_u = 40$ | $\rho = 1.10$ | $\rho = 1.60$ | High var. | $c^r = 3$ | 30% LIFO | Real data |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| vs LA-Quantile | **-4.9** | **-5.0** | **-3.9** | **+2.1** | **-2.5** | -0.7 | +0.0 | -0.8 | **-2.4** | **-1.7** | -0.4 | **-4.9** | -1.4 |
| vs LA-Quantile2 | **+1.0** | **+1.9** | **+2.8** | **+5.7** | +0.3 | **+1.4** | +0.7 | **+1.8** | +0.8 | **+3.3** | **+2.4** | **+1.5** | +1.5 |
| vs ScenPO-SS | **-3.4** | **-4.5** | **-5.2** | **+9.8** | **-6.9** | **-5.9** | +2.2 | **+5.5** | **-16.0** | **-3.7** | -1.1 | -1.8 | +2.0 |
| vs LA-Robust | **-3.4** | **-2.9** | -1.1 | **+4.8** | **-1.8** | **+0.9** | +1.9 | -0.2 | -0.3 | **+1.8** | +0.5 | -1.2 | -0.6 |
| vs BSP-EW | **+6.0** | **+11.8** | **+8.1** | **+12.7** | **+5.1** | **+1.9** | **+15.2** | **+8.6** | **+2.2** | **+6.8** | **-3.3** | **+6.2** | **+10.5** |
| vs $UL_\lambda$ | **+7.5** | **+12.1** | **+8.6** | **+13.4** | **+7.2** | **+3.5** | **+16.8** | **+9.8** | **+4.1** | **+8.8** | **+1.5** | **+8.1** | **+11.7** |

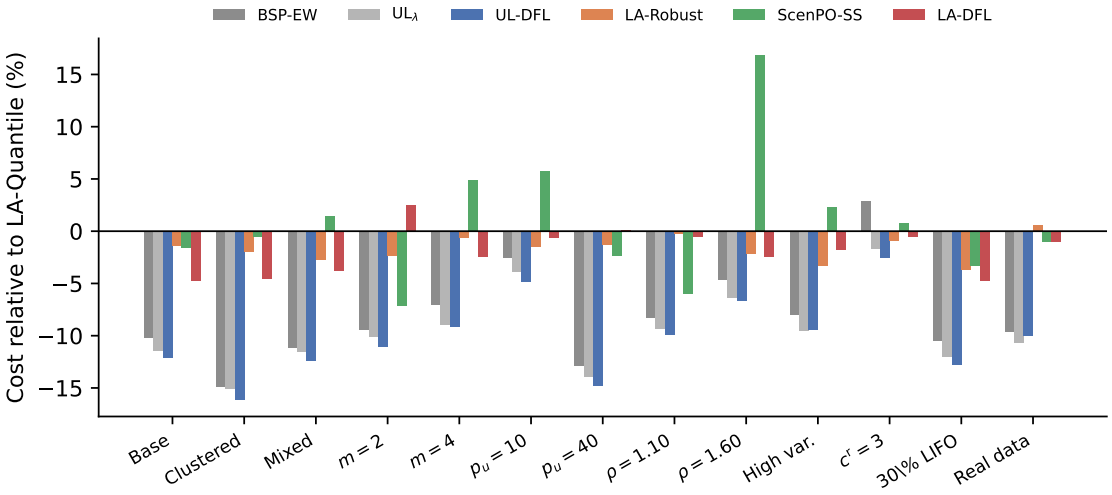



Figure 4: Cost of selected policies relative to LA-Quantile, by configuration (earlier test episodes; LA-Mean is omitted for readability).

### 6.6 Real-data case

On observed demand (column "Real data" of Tables 9 and 10) the ranking of the policy families is the same as in the synthetic base case, but decision-focused learning of the order-up-to level does not pay. The re-tuned UL$_\lambda$ is the cheapest policy (12,937 per episode), and UL-DFL is 0.6% more expensive (95% CI [12, 163]). Without learning, the rules (12,937–13,090) are again cheaper than every integrated policy, the best of which, LA-Quantile2, costs 14,185, and LA-Mean is again the most expensive. Service is lower than in the synthetic study for all policies (fill rate 93.9% for UL$_\lambda$), because observed demand is more volatile and its promotion and weekend peaks occasionally exceed the daily fleet capacity.

The failure of UL-DFL is one of transfer, not of training. On the validation episodes all three networks improved on the rule, by 0.6–0.9%, and model selection therefore kept them; on the test episodes all three are more expensive than the rule (Table 8). Validation and test episodes come from different products (28 and 56 products), and the corrections the networks learn are tied to the demand patterns of the 101 training products. They are more than twice as large as in the synthetic base case (standard deviation of $\delta_\theta$ 0.73 against 0.30) and follow a pronounced weekly pattern: the order-up-to level is raised on Friday (implied service level 0.97) and lowered from Saturday to Monday (0.87, 0.80 and 0.87). For the test products this removes some waste (waste rate 3.40% against 3.64%) but costs more in lost sales (fill rate 93.6% against 93.9%). Validation-based model selection protects against harmful corrections only when the validation episodes are representative of those on which the policy is used; with few products and strong product-specific patterns, a larger validation set or a stronger penalty on deviations from the rule would be needed. LA-DFL is 1.4% cheaper than LA-Quantile, which is not significant (Holm-adjusted $p = 0.226$), and LA-Quantile2 is again cheaper than LA-DFL, though not significantly.

### 6.7 Store-level service

Average fill rates hide how service is distributed over the network (Figure 5). In all three layouts the inventory rules, with or without learning, serve every store at least 95.4%. Their orders do not depend on where a store is located, and neither does their service. The deterministic lookahead policies (LA-Quantile, LA-Quantile2, LA-Robust, LA-DFL) do not serve the stores alike: their worst-served store reaches only 89.1–90.3% in the base layout (fresh test episodes)

and 89.6–94.5% in the clustered and mixed layouts. What matters is not the distance from the depot but the travel a visit adds relative to what the store sells. We measure it by the detour of inserting the store into a tour through all stores, divided by its mean daily demand. In the base and mixed layouts one store stands out on this measure (1.3 distance units per unit of demand, against at most 0.8 for every other store), and in both layouts it is the worst-served store of every deterministic lookahead policy (89.1–93.6%), whereas the rules, with or without learning, serve it at 96.1–96.8%. Consolidating deliveries saves the most travel at such a store, and the plan then accepts shortages there. In the clustered layout, where every detour is short (at most 0.3), the shortfall of the lookahead policies is smaller (worst store 92.3–94.5%). With ten stores per layout this is an observation about individual stores rather than an estimated relationship. ScenPO-SS serves every store less well (worst store 90.9–91.9% across the layouts), but its shortfall is not concentrated at the expensive store (92.0–92.9% there). This effect is relevant for retailers with service-level agreements per store: an integrated plan may concentrate stock-outs at individual stores even when its average fill rate is high.

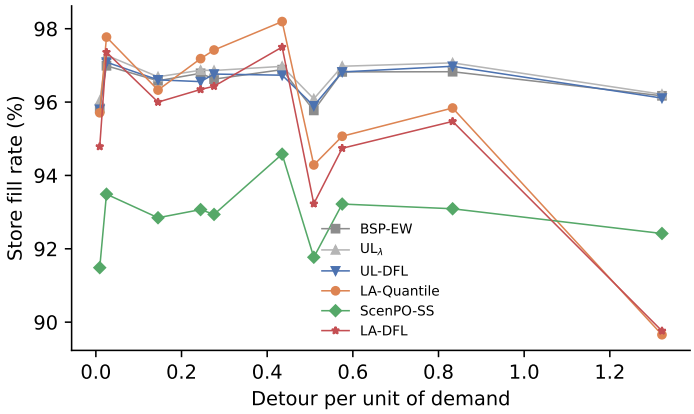



Figure 5: Store fill rate against the store's detour per unit of demand, i.e. the extra travel of inserting the store into a tour through all stores divided by its mean daily demand (base case, mean over 100 fresh test episodes).

## 6.8 Computation

All 14,700 lookahead decisions on the fresh base-case test episodes were solved to a proven gap of at most 0.50%, with 0 time-limit stops and 0 fallbacks; the same holds in every configuration reported. With 12 decisions solved in parallel, a deterministic lookahead decision takes about 1.08 seconds and a ScenPO-SS decision with 20 scenarios 3.41 seconds; the rules, with or without learned corrections, need milliseconds. Training one UL-DFL network (30 epochs over 200 episodes, including closed-loop validation after every epoch) took about two minutes; training one LA-DFL network (2,100 states, eight epochs including MILP re-solves and closed-loop validation) about 45 minutes. The complete base case, including tuning of all policies and three seeds of each learner, took 4.2 hours, and each other configuration 1.0–2.6 hours with 12–15 parallel workers (training three NN-DFL networks adds about ten minutes).

The lookahead policies are deterministic: solving the same decision twice returns the same plan. Their closed-loop cost is nevertheless sensitive to negligible changes in the planning input, because several plans lie within the 0.5% optimality gap and the solver returns the first one it proves; a different plan on one day changes the stock and therefore every later decision. To quantify this noise floor we re-ran LA-Quantile on the earlier test episodes with its planning targets rounded to single precision, a relative change below $10^{-7}$. The plan changed on at least one day in each of the 1,300 test episodes of the thirteen configurations, and episode costs moved by 3.5% (standard deviation) in the base case, between -9.7% and +11.2%, and by 1.6–5.0%

across configurations. Averaged over 100 episodes these changes largely cancel (mean change at most 0.48% in absolute value, standard error at most 0.50%). Paired comparisons remain valid, because this variation is part of the episode-level noise they account for, but differences of well below one percent between two lookahead policies should not be interpreted as a systematic effect. The rule-based policies, including UL-DFL, do not solve an optimisation model for their orders and are not affected.

### 6.9 Verification of the structural results

Table 12 reports numerical checks of the results in Section 3.3 (script `verify_theory.py` in the replication package). Theorem 1(ii) is checked by solving the recourse a second time with FIFO issuing and greedy service imposed through binary indicator constraints: if the theorem were false, the constrained value would exceed $V_{\mathrm{LP}}$ in some instance. The bound of Proposition 2(b) is attained in 71% of the random instances, so it cannot be tightened in general. The physical FIFO cost equals $V_{\mathrm{LP}}$ in 63% of the random instances; the remaining gap is the value of discarding stock early, which a store does not do in practice.

Table 12: Numerical verification of the structural results.

| Result | Check | Outcome |
|---|---|---|
| Theorem 1(i) | 60 lookahead MILPs ($m \in \{2,3,4\}$, 1 or 5 scenarios) with demand from zero to four times fleet capacity | all feasible, all solved to proven gap |
| Theorem 1(ii) | 3,000 random recourse instances ($m \in \{2,3,4\}$, $H \in \{2,\dots,5\}$, random costs); FIFO- and greedy-constrained MILP value vs. $V_{\mathrm{LP}}$ | max. absolute difference $10^{-6}$ |
| Theorem 1(iii) | $V_{\mathrm{LP}} \le C_{\mathrm{FIFO}}$ in the same instances | holds in 3,000/3,000; equality in 63% |
| Proposition 2(a) | dual bounds $[-p_u, p_s]$; subgradient inequality in 9,000 random directions | 3,000/3,000; 9,000/9,000 |
| Proposition 2(b) | autograd derivative of $C_{\mathrm{FIFO}}$ within (7) | 3,000/3,000; bound attained in 71% |
| Proposition 3(c) | $m = 1$, negative-binomial demand (mean 15), SGD on the decision-focused loss (with $m = 1$ both learners deliver the learned quantity, so the check covers (a) and (b)) | learned target 20.03; empirical 0.8-quantile 20.0; $F(\tilde{d}) = 0.809$ vs. $p_u/(p_u + p_s) = 0.8$ |
| Coverage layer | 40 lookahead MILPs; deliveries implied by the visit pattern vs. MILP deliveries | 98.7% of delivered volume reproduced (mean relative $L_1$ deviation 1.3%) |

## 7 Discussion and conclusion

### 7.1 Managerial implications

The study asked how contextual forecasts should be turned into replenishment decisions when perishable stock ages and deliveries share vehicles. Five implications follow for planners of fresh-food distribution.

1. **Start from a stock-age-based replenishment rule, tune it finely, and benchmark against it before integrating.** Daily replenishment at a high service quantile, with the orders routed optimally, was cheaper than every integrated inventory-routing policy in all settings but the one with expensive routing, by 2.0–11.9%, across three network layouts, shelf lives of two to four days, lower and higher lost-sales penalties, tighter and looser fleets, more

variable demand, partial LIFO issuing and observed demand of a fresh-food retailer; the margin was smallest when lost sales were cheap, so that high service is worth less. In these settings the travel saved by consolidating deliveries is smaller than the extra holding, waste and lost sales that consolidation causes. A finer grid for the service level and the visit trigger alone lowered the cost of the deliver-up-to-level rule by up to 0.6%. This extends the finding of Haijema and Minner [27] and Crama et al. [17] that well-tuned simple rules are hard to beat to a setting with shared routes and contextual forecasts.

2. **Learn the order-up-to level, not only the forecast, but check that what is learned carries over.** Training the order-up-to level of the deliver-up-to-level rule on the realised cost of its decisions reduced cost by about 1% in nine of thirteen settings and on fresh test episodes of the base case, where it reached almost the same fill rate with 28% less waste, which no re-tuning of the rule's service level and trigger achieves. The gain was smallest, and not significant, where the rule already wasted little (four-day shelf life), with a loose fleet, and where demand was too variable for the context to anticipate it. On observed demand of new products the learned corrections did not carry over, and the rule was cheaper: corrections should be validated on products and periods like those on which they will be used, and the rule kept where they are not. The learning also needs the rule: the same network trained to order directly, or to set its own order-up-to levels, was more expensive than UL-DFL in twelve settings. The policy is operated exactly like the rule: it remains a rule with a visit trigger and an order-up-to level that planners can inspect. The learned corrections translate into practice that does not require machine learning: shift safety stock from the days after the weekly peak to the days before it, and, where promotions widen the forecast spread, lower the order-up-to level on promotion days.

3. **Integration pays when routing is expensive, and then the safety stock of the plan matters most.** With three times higher travel cost, a lookahead with day-specific quantile levels was the cheapest policy, level with UL-DFL, and every uncertainty-aware lookahead beat daily base-stock. Scenario-based lookahead with the scenario set of Jia et al. [37], which does not extend beyond the 0.9 quantile of the forecasts, under-stocked; with scenarios drawn from bands of the full quantile grid, which reach further into the upper tail, it would have been the cheapest policy in that setting. Deterministic lookahead with mean forecasts was the worst policy in every setting.

4. **If a deterministic plan is used, its safety stock should depend on the planning day.** Decision-focused learning on top of the lookahead learned to keep a high service quantile only for the day that is actually executed and to plan later days near the 0.8 quantile. A lookahead with these two quantile levels, tuned on validation data, removes about half of the waste of the single-quantile plan in the base case and was at least as cheap as the learned targets in all thirteen settings.

5. **Look at store-level service, not only at averages.** Integrated deterministic plans concentrated stock-outs at the store whose visit costs the most travel per unit sold, whereas the inventory rules served all stores alike. Retailers with per-store service targets should constrain or monitor this effect.

### 7.2 Limitations and further research

The synthetic demand is generated by a documented contextual model; the real-data case uses observed sales of one retailer, from which latent demand had to be reconstructed under stock-outs and rescaled to the network's capacities, and from which products whose level shifted strongly were excluded. The test episodes of the synthetic configurations had been evaluated in an earlier round of the study; the primary hypotheses are therefore tested on fresh test episodes of the

base case, and the other synthetic configurations are secondary evidence. In the real-data case, model selection on validation episodes did not protect against corrections that failed to transfer to new products; regularising the corrections towards the rule, or validating across products and periods, are natural remedies. In the synthetic study the forecasts are built from uncensored demand histories; with censored sales, demand estimates are biased downwards where stock-outs occur, which would affect the policies with the lowest fill rates most [48]. The holding and waste costs are not varied, only the lost-sales penalty. Exact routing by enumeration limits the network to about a dozen stores; larger networks require heuristic routing, which would weaken the clean separation between replenishment and routing quality used here. Lead time is zero and each store sells one product; positive lead times would increase the value of stock-age information [11], and multiple products would couple stores through shared vehicle space. The lookahead policies relax routing integrality beyond the implemented day, which favours visiting stores more often than a fully integer lookahead would, and they give stock left at the end of the horizon no terminal value; a longer horizon of four or five days lowered the cost of the best lookahead by about 1% in the base case without changing the ranking. ScenPO-SS was implemented with the scenario set of the original study; its cost depends on how far the scenarios reach into the upper tail of the forecast, and tuning this choice on validation data, as done for the quantile levels of the deterministic lookahead, could make it more competitive. UL-DFL learns the order-up-to level but keeps the rule's visit trigger and sees fleet capacity only as a proportional scaling during training; learning the trigger as well, or training against the exact routing cost, could extend its advantage to settings with expensive routing. LA-DFL was trained on the decision states visited by LA-Quantile in one round; iterating between training and data collection, or training the scenario generator of ScenPO-SS decision-focusedly, are natural extensions. Finally, deep reinforcement learning has shown promise for single-location perishable inventory [18, 54]; extending it to the routing-coupled action space studied here is an open problem.

### 7.3 Conclusion

Replenishing perishables on shared routes is a perishable inventory problem first and a routing problem second. Under a common closed-loop simulator with exact optimisation, stock-age-based replenishment rules were the strongest policies without learning unless travel was expensive, in which case a lookahead with day-specific safety quantiles matched them. Decision-focused learning made a well-tuned rule about 1% cheaper and markedly less wasteful in most settings, including fresh test episodes, by learning from realised cost when to deviate from a fixed service quantile; on observed demand of new products its corrections did not carry over. On top of a deterministic lookahead it revealed why such plans mis-stock, a lesson that a day-specific quantile level implements without learning. The structural results, the experimental protocol and the code are available for testing further policies on the same benchmark.

## Data and code availability

All code, configuration files, random seeds and result files needed to reproduce every table and figure are provided in the replication package (`pirp/`, `experiments/`, `results_v2/`). The synthetic demand data are generated by the package. The real-data case uses the public FreshRetailNet-50K dataset [57] (CC BY 4.0), which is downloaded from Hugging Face as described in the package and then processed by its code; no proprietary data are used.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic), through the Claude Code assistant, to help write and restructure the simulation and experiment code, run the experiments, and draft and edit the text. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

## References


[1] Aldair Alvarez, Jean-François Cordeau, Raf Jans, Pedro Munari, and Reinaldo Morabito. Formulations, branch-and-cut and a hybrid heuristic algorithm for an inventory routing problem with perishable products. *European Journal of Operational Research*, 283(2): 511–529, 2020. doi: 10.1016/j.ejor.2019.11.015.

[2] Matias Alvo, Daniel Russo, and Yash Kanoria. Neural inventory control in networks via hindsight differentiable policy optimization. arXiv preprint arXiv:2306.11246, 2023.

[3] Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 136–145. PMLR, 2017. URL http://proceedings.mlr.press/v70/amos17a.html.

[4] Monique Bakker, Jan Riezebos, and Ruud H. Teunter. Review of inventory systems with deterioration since 2001. *European Journal of Operational Research*, 221(2):275–284, 2012. doi: 10.1016/j.ejor.2012.03.004.

[5] Gah-Yi Ban and Cynthia Rudin. The big data newsvendor: Practical insights from machine learning. *Operations Research*, 67(1):90–108, 2019. doi: 10.1287/opre.2018.1757.

[6] Luca Bertazzi, Adamo Bosco, Francesca Guerriero, and Demetrio Laganà. A stochastic inventory routing problem with stock-out. *Transportation Research Part C: Emerging Technologies*, 27:89–107, 2013. doi: 10.1016/j.trc.2011.06.003.

[7] Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In *Advances in Neural Information Processing Systems*, volume 33, pages 9008–9019, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/66808e327dc79d135ba18e051673d906-Abstract.html.

[8] Dimitris Bertsimas and Aurélie Thiele. A robust optimization approach to inventory theory. *Operations Research*, 54(1):150–168, 2006. doi: 10.1287/opre.1050.0238.

[9] Marco Bijvank and Iris F. A. Vis. Lost-sales inventory theory: A review. *European Journal of Operational Research*, 215(1):1–13, 2011. doi: 10.1016/j.ejor.2011.02.004.

[10] Robert N. Boute, Joren Gijsbrechts, Willem van Jaarsveld, and Nathalie Vanvuchelen. Deep reinforcement learning for inventory control: A roadmap. *European Journal of Operational Research*, 298(2):401–412, 2022. doi: 10.1016/j.ejor.2021.07.016.

[11] Rob A. C. M. Broekmeulen and Karel H. van Donselaar. A heuristic to manage perishable inventory with batch ordering, positive lead-times, and time-varying demand. *Computers & Operations Research*, 36(11):3013–3018, 2009. doi: 10.1016/j.cor.2009.01.017.

[12] Jinzhi Bu, Xiting Gong, and Xiuli Chao. Asymptotic optimality of base-stock policies for perishable inventory systems. *Management Science*, 69(2):846–864, 2023. doi: 10.1287/mnsc.2022.4400.

[13] Xiuli Chao, Xiting Gong, Cong Shi, and Huanan Zhang. Approximation algorithms for perishable inventory systems. *Operations Research*, 63(3):585–601, 2015. doi: 10.1287/opre.2015.1386.

[14] Leandro C. Coelho and Gilbert Laporte. Optimal joint replenishment, delivery and inventory management policies for perishable products. *Computers & Operations Research*, 47:42–52, 2014. doi: 10.1016/j.cor.2014.01.013.

[15] Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. Heuristics for dynamic and stochastic inventory-routing. *Computers & Operations Research*, 52:55–67, 2014. doi: 10.1016/j.cor.2014.07.001.

[16] Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. Thirty years of inventory routing. *Transportation Science*, 48(1):1–19, 2014. doi: 10.1287/trsc.2013.0472.

[17] Yves Crama, Mahmood Rezaei, Martin W. P. Savelsbergh, and Tom van Woensel. Stochastic inventory routing for perishable products. *Transportation Science*, 52(3):526–546, 2018. doi: 10.1287/trsc.2017.0799.

[18] Bram J. De Moor, Joren Gijsbrechts, and Robert N. Boute. Reward shaping to improve the performance of deep reinforcement learning in perishable inventory management. *European Journal of Operational Research*, 301(2):535–545, 2022. doi: 10.1016/j.ejor.2021.10.045.

[19] Priya L. Donti, Brandon Amos, and J. Zico Kolter. Task-based end-to-end model learning in stochastic optimization. In *Advances in Neural Information Processing Systems*, volume 30, pages 5484–5494, 2017. doi: 10.48550/arXiv.1703.04529.

[20] Adam N. Elmachtoub and Paul Grigas. Smart "predict, then optimize". *Management Science*, 68(1):9–26, 2022. doi: 10.1287/mnsc.2020.3922.

[21] Adam N. Elmachtoub, Henry Lam, Haofeng Zhang, and Yunfan Zhao. Estimate-then-optimize versus integrated-estimation-optimization versus sample average approximation: A stochastic dominance perspective. arXiv preprint arXiv:2304.06833, 2023.

[22] Awi Federgruen, Gregory Prastacos, and Paul H. Zipkin. An allocation and distribution model for perishable products. *Operations Research*, 34(1):75–82, 1986. doi: 10.1287/opre.34.1.75.

[23] Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022. doi: 10.1287/msom.2021.1064.

[24] Toni Greif, Louis Bouvier, Christoph M. Flath, Axel Parmentier, Sonja U. K. Rohmer, and Thibaut Vidal. Combinatorial optimization and machine learning for dynamic inventory routing. arXiv preprint arXiv:2402.04463, 2024.

[25] René Haijema. Optimal ordering, issuance and disposal policies for inventory management of perishable products. *International Journal of Production Economics*, 157:158–169, 2014. doi: 10.1016/j.ijpe.2014.06.014.

[26] René Haijema and Stefan Minner. Stock-level dependent ordering of perishables: A comparison of hybrid base-stock and constant order policies. *International Journal of Production Economics*, 181:215–225, 2016. doi: 10.1016/j.ijpe.2015.10.013.

[27] René Haijema and Stefan Minner. Improved ordering of perishables: The value of stock-age information. *International Journal of Production Economics*, 209:316–324, 2019. doi: 10.1016/j.ijpe.2018.03.008.

[28] Zhichao He, Ya Liu, and Kun Liu. Robust optimization approaches for the perishable product inventory routing problem with demand uncertainty. *Journal of Industrial and Management Optimization*, 20(8):2740–2769, 2024. doi: 10.3934/jimo.2024024.

[29] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962. doi: 10.1137/0110015.

[30] Vera Hemmelmayr, Karl F. Doerner, Richard F. Hartl, and Martin W. P. Savelsbergh. Delivery strategies for blood products supplies. *OR Spectrum*, 31(4):707–725, 2009. doi: 10.1007/s00291-008-0134-7.

[31] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. doi: 10.2307/4615733.

[32] Yichun Hu, Nathan Kallus, and Xiaojie Mao. Fast rates for contextual linear optimization. *Management Science*, 68(6):4236–4245, 2022. doi: 10.1287/mnsc.2022.4383.

[33] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[34] Jakob Huber, Sebastian Müller, Moritz Fleischmann, and Heiner Stuckenschmidt. A data-driven newsvendor problem: From data to decision. *European Journal of Operational Research*, 278(3):904–915, 2019. doi: 10.1016/j.ejor.2019.04.043.

[35] MD Shafikul Islam Sohan and Azmine Toushik Wasi. Optimizing inventory routing: A decision-focused learning approach using neural networks. In *New in Machine Learning Workshop, NeurIPS 2023 (Extended Abstracts)*, 2023. URL https://openreview.net/forum?id=r0fzjB8f7f. Workshop extended abstract; arXiv:2311.00983.

[36] Larissa Janssen, Thorsten Claus, and Jürgen Sauer. Literature review of deteriorating inventory models by key topics from 2012 to 2015. *International Journal of Production Economics*, 182:86–112, 2016. doi: 10.1016/j.ijpe.2016.08.019.

[37] Menglei Jia, Albert H. Schrotenboer, and Feng Chen. Scenario predict-then-optimize for data-driven online inventory routing. *Transportation Science*, 59(5):1032–1056, 2025. doi: 10.1287/trsc.2024.0613.

[38] Illya Kaynov, Marijn van Knippenberg, Vlado Menkovski, Albert van Breemen, and Willem van Jaarsveld. Deep reinforcement learning for one-warehouse multi-retailer inventory management. *International Journal of Production Economics*, 267:109088, 2024. doi: 10.1016/j.ijpe.2023.109088.

[39] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[40] Anton J. Kleywegt, Vijay S. Nori, and Martin W. P. Savelsbergh. The stochastic inventory routing problem with direct deliveries. *Transportation Science*, 36(1):94–118, 2002. doi: 10.1287/trsc.36.1.94.574.

[41] Sebastian Malicki and Stefan Minner. Cyclic inventory routing with dynamic safety stocks under recurring non-stationary interdependent demands. *Computers & Operations Research*, 131:105247, 2021. doi: 10.1016/j.cor.2021.105247.

[42] Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Víctor Bucarey, Tias Guns, and Ferdinando Fioretto. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *Journal of Artificial Intelligence Research*, 80:1623–1701, 2024. doi: 10.1613/jair.1.15320.

[43] Stefan Minner and Sandra Transchel. Periodic review inventory-control for perishable products under service-level constraints. *OR Spectrum*, 32(4):979–996, 2010. doi: 10.1007/s00291-010-0196-1.

[44] Samira Mirzaei and Abbas Seifi. Considering lost sale in inventory routing problems for perishable goods. *Computers & Industrial Engineering*, 87:213–227, 2015. doi: 10.1016/j.cie.2015.05.010.

[45] Maxime Mulamba, Jayanta Mandi, Michelangelo Diligenti, Michele Lombardi, Victor Bucarey, and Tias Guns. Contrastive losses and solution caching for predict-and-optimize. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)*, pages 2833–2840, 2021. doi: 10.24963/ijcai.2021/390.

[46] Steven Nahmias. Perishable inventory theory: A review. *Operations Research*, 30(4):680–708, 1982. doi: 10.1287/opre.30.4.680.

[47] Warren B. Powell. A unified framework for stochastic optimization. *European Journal of Operational Research*, 275(3):795–821, 2019. doi: 10.1016/j.ejor.2018.07.014.

[48] Anna-Lena Sachs and Stefan Minner. The data-driven newsvendor with censored demand observations. *International Journal of Production Economics*, 149:28–36, 2014. doi: 10.1016/j.ijpe.2013.04.039.

[49] Utsav Sadana, Abhilash Chenreddy, Erick Delage, Alexandre Forel, Emma Frejinger, and Thibaut Vidal. A survey of contextual optimization methods for decision-making under uncertainty. *European Journal of Operational Research*, 320(2):271–289, 2025. doi: 10.1016/j.ejor.2024.03.020.

[50] Marius M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987. doi: 10.1287/opre.35.2.254.

[51] Oğuz Solyalı, Jean-François Cordeau, and Gilbert Laporte. Robust inventory routing under demand uncertainty. *Transportation Science*, 46(3):327–340, 2012. doi: 10.1287/trsc.1110.0387.

[52] Mehmet Soysal, Jacqueline M. Bloemhof-Ruwaard, René Haijema, and Jack G. A. J. van der Vorst. Modeling an inventory routing problem for perishable products with environmental considerations and demand uncertainty. *International Journal of Production Economics*, 164:118–133, 2015. doi: 10.1016/j.ijpe.2015.03.008.

[53] Bo Tang and Elias B. Khalil. PyEPO: A PyTorch-based end-to-end predict-then-optimize library for linear and integer programming. *Mathematical Programming Computation*, 16(3):297–335, 2024. doi: 10.1007/s12532-024-00255-x.

[54] Tarkan Temizöz, Christina Imdahl, Remco Dijkman, Douniel Lamghari-Idrissi, and Willem van Jaarsveld. Deep controlled learning for inventory control. *European Journal of Operational Research*, 324(1):104–117, 2025. doi: 10.1016/j.ejor.2025.01.026.

[55] Karel van Donselaar, Tom van Woensel, Rob Broekmeulen, and Jan Fransoo. Inventory control of perishables in supermarkets. *International Journal of Production Economics*, 104(2):462–472, 2006. doi: 10.1016/j.ijpe.2004.10.019.

[56] Karel H. van Donselaar and Rob A. C. M. Broekmeulen. Approximations for the relative outdating of perishable products by combining stochastic modeling, simulation and regression modeling. *International Journal of Production Economics*, 140(2):660–669, 2012. doi: 10.1016/j.ijpe.2012.02.023.

[57] Yangyang Wang, Jiawei Gu, Li Long, Xin Li, Li Shen, Zhouyu Fu, Xiangjun Zhou, and Xu Jiang. FreshRetailNet-50K: A stockout-annotated censored demand dataset for latent demand recovery and forecasting in fresh retail. arXiv preprint arXiv:2505.16319, 2025. Dataset: https://huggingface.co/datasets/Dingdong-Inc/FreshRetailNet-50K, CC BY 4.0.

# A Proofs

*Proof of Proposition 1.* Fix $v$ and $k$ and let $R^\star = \{i : z_{ivk} = 1\}$. Constraint (3) gives $\sum_R y_{Rvk} = 1$ and $\sum_{R \ni i} y_{Rvk} = z_{ivk}$. For $i \in R^\star$ the second equality equals the first, so every $R$ with $y_{Rvk} > 0$ contains $i$; for $i \notin R^\star$ the sum is zero, so no such $R$ contains $i$. Hence $y_{Rvk} > 0$ implies $R = R^\star$, and $y_{R^\star vk} = 1$. The routing cost is $\mathrm{TSP}(R^\star)$, vehicle loads are at most $Q$ by (4), and each store is on at most one vehicle. Every feasible lookahead plan (stores per vehicle, loads, inventory flows) has exactly one representation of this form, which proves exactness. □

*Proof of Theorem 1.* (i) Setting $z = 0$, $y_{\emptyset vk} = 1$, $q = 0$, $\zeta = 0$ and serving demand from available stock in any order is feasible: day-0 stock satisfies the shelf constraint by assumption and stock only decreases afterwards. All cost terms (travel, holding of carried units, waste, lost sales) are non-negative, so the objective is bounded below by zero; with finitely many assignments and a bounded linear programme for each, an optimum exists.

(ii) Take an optimal recourse solution and view inventory as fungible flows of units per cohort. *Greedy service.* If on day $k$ demand is lost while $\varepsilon > 0$ units are left on the shelf, sell $\varepsilon$ of them. Those units originally had one of four fates: sold on a later day (that later sale is now lost; the cost change is $-p_u\varepsilon + p_u\varepsilon - h\varepsilon \cdot (\text{nights saved}) \le 0$), expired or disposed (saving $p_s\varepsilon$ and holding), or carried beyond the horizon (saving holding). The cost does not increase. *FIFO.* If on day $k$ a younger cohort $r'$ is sold while $\varepsilon$ units of an older cohort $r < r'$ are left, sell $\varepsilon$ older units instead and keep $\varepsilon$ younger ones. The kept younger units replicate the original path of the older units: they are sellable on every day the older units were (their remaining life is longer); where the older units expired at the end of day $k + r - 1$, the younger units are disposed on that day at the same cost $p_s$ and with the same number of holding nights; disposal and carrying beyond the horizon are replicated identically. The number of units carried each night is unchanged, so the cost is unchanged. Each exchange reduces $\sum_k \sum_r r\, w_{irk}$ and the feasible set is a polytope, so a finite sequence of exchanges yields an optimal solution with FIFO issuing and greedy service.

(iii) $q$ enters the recourse LP only through the right-hand side of $S_{imk} = q_k$, so $V_{\mathrm{LP}}$ is the value function of an LP in its right-hand side, which is convex and piecewise linear. The physical FIFO trajectory with $\zeta = 0$ is feasible for the recourse LP and has cost $C_{\mathrm{FIFO}}$; hence $V_{\mathrm{LP}} \le C_{\mathrm{FIFO}}$. By (ii) there is an optimal solution with FIFO issuing and greedy service; if it uses no disposal it coincides with the physical trajectory and the two values are equal. □

*Proof of Proposition 2.* (a) By LP duality, the optimal dual of the constraint $S_{imk} = q_k$ is a subgradient of the value function with respect to $q_k$. Adding $\varepsilon$ to $q_k$ and disposing of these units immediately is feasible and costs $p_s\varepsilon$ (holding $h\varepsilon$ through $S_{imk}$ plus $(p_s - h)\varepsilon$ through $\zeta$), so $V_{\mathrm{LP}}(q + \varepsilon e_k) \le V_{\mathrm{LP}}(q) + p_s\varepsilon$ and $\pi_k \le p_s$. Removing $\varepsilon \le q_k$ units turns at most $\varepsilon$ sales into lost sales and cannot increase holding or waste, so $V_{\mathrm{LP}}(q - \varepsilon e_k) \le V_{\mathrm{LP}}(q) + p_u\varepsilon$ and $\pi_k \ge -p_u$.

(b) $C_{\mathrm{FIFO}}$ is a composition of finitely many min, max and affine maps of $q$, hence piecewise linear and Lipschitz. Compare the FIFO trajectories with deliveries $q$ and $q + \varepsilon e_k$. On day $k$ the

extra units are the freshest and are sold last, so sales of older stock are unchanged. On later days the extra units are older than subsequent deliveries and may displace them, but stock that was older than the extra units is never affected. Therefore, relative to the original trajectory, lost sales can only fall, waste and carried stock can only rise, and the total extra stock $E_\ell$ carried on night $\ell$ satisfies $0 \le E_\ell \le \varepsilon$. Each extra unit leaves the system by preventing a lost sale ($-p_u$), by expiring on some day $\tau \le H-1$ ($+p_s$ after at most $\tau - k \le H-1-k$ nights), or by remaining at the end ($H-k$ nights). The cost change per unit is therefore at least $-p_u$ and at most $\max\{p_s + (H-1-k)h, (H-k)h\} = p_s + (H-1-k)h$ when $h \le p_s$. Dividing by $\varepsilon$ gives (7) for both one-sided derivatives. □

*Proof of Proposition 3.* (a) With the visit pattern fixed and no capacity binding, the coverage map gives $q_k = \sum_{j \in W_k} \tilde{d}_j - U_k(\tilde{d})$, where $U_k$ is the planned usable stock that serves the window. A unit increase of $\tilde{d}_j$, $j \in W_k$, increases $q_k$ by one unless the planned stock already covers day $j$, in which case the target of day $j$ does not affect any delivery; days outside every window affect no delivery. The chain rule then gives (8), and setting it to zero is the first-order condition of $\min_q \mathbb{E}[C_{\mathrm{FIFO}}(q; D) \mid \xi]$ over the deliveries of the fixed pattern.

(b) With the trigger pattern fixed and no capacity binding, a triggered delivery is $q_{it} = \tilde{S}_{it} - U_{it}$, where the usable stock $U_{it}$ depends on earlier deliveries and demands but not on $\tilde{S}_{it}$. Hence $\partial q_{it}/\partial \tilde{S}_{it} = 1$, and $\tilde{S}_{it}$ affects the episode only through $q_{it}$; later deliveries respond to it through their usable stock, with their own levels held fixed. The chain rule gives (9), with $\mathrm{d}C^T/\mathrm{d}q_{it}$ the total derivative along the trajectory. If $\delta_\theta$ can take any value for each input, stationarity of $\mathbb{E}[C^T]$ in $\theta$ implies that (9) vanishes for every input, which is the first-order condition of minimising $\mathbb{E}[C^T]$ over the order-up-to levels of the triggered store-days given their inputs.

(c) With $m = 1$ nothing is carried: $W_k = \{k\}$ and $q_k = \tilde{d}_k$ in (a), $U_{it} = 0$ and $q_{it} = \tilde{S}_{it}$ in (b), and the cost of a day, $p_u(D-q)^+ + p_s(q-D)^+$, does not depend on other days. Thus $\mathbb{E}[\partial C/\partial q \mid \xi] = -p_u \mathbb{P}(D > q \mid \xi) + p_s \mathbb{P}(D < q \mid \xi)$, which vanishes when $F_{D|\xi}(q) = p_u/(p_u + p_s)$ (for continuous $F$; for discrete demand, at the smallest $q$ with $F \ge p_u/(p_u + p_s)$). □

# B First versions of the rule tuning and of ScenPO-SS, and the rule as published

Table 13 compares, on the earlier test episodes of every configuration, the first versions of three policies with the versions reported in the main text. $\mathrm{UL}_\lambda$ tuned on the fine grid is up to 0.6% cheaper than when tuned on the coarse grid, significantly in six configurations. About half of the advantage of the first version of UL-DFL, trained from the coarse rule, therefore came from the resolution of the grid: against the rule tuned on the fine grid it was significantly cheaper in six configurations and more expensive with a four-day shelf life. Retrained from the fine rule, UL-DFL is 0.8–1.2% cheaper than it in nine configurations. ScenPO-SS with the scenario set of Jia et al. [37] is more expensive than the first implementation in twelve configurations (significantly in nine) and cheaper with a two-day shelf life. The first implementation sampled ten scenarios with equal probability from the twelve bands of the full quantile grid; three of these bands lie above the 0.9 quantile, where only 9% of the probability mass lies, so its scenarios over-represented high demand. It served 92.0–97.1% of demand, against 87.2–94.7% for the published scenario set, and with expensive routing it would have been the cheapest policy (18,197 per episode against 18,630 for LA-Quantile2). On the fresh base-case test episodes the two versions of ScenPO-SS differ by 0.8%, which is not significant.

The two rows on the published rule compare $\mathrm{UL}_\lambda$ with the deliver-up-to-level rule exactly as published by Crama et al. [17]: one target service level for both the trigger and the delivery quantity, the total stock counted, and $\lambda \in \{1, m-1\}$, with the service level tuned on the same validation episodes over the fine grid (the paper's 90% is in the grid). The published rule is significantly more expensive than our $\mathrm{UL}_\lambda$ in every configuration, by 0.6–3.4% (1.6% on the

fresh base-case test episodes), and UL-DFL is 0.9–3.8% cheaper than it. Tuning the two service levels separately and netting out the stock about to expire therefore make the rule a stronger comparator, not a weaker one.

Table 13: First versus final versions, and the rule as published, on the earlier test episodes: paired cost difference of the first-named policy relative to the second, in percent. Bold: 95% paired bootstrap interval excludes zero.

| Comparison (%) | Base | Clustered | Mixed | $m = 2$ | $m = 4$ | $p_u = 10$ | $p_u = 40$ | $\rho = 1.10$ | $\rho = 1.60$ | High var. | $c^r = 3$ | 30% LIFO | Real data |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\mathrm{UL}_\lambda$, fine vs. coarse grid | **-0.5** | +0.1 | +0.1 | -0.3 | **-0.6** | +0.0 | -0.6 | **-0.5** | **-0.6** | -0.4 | -0.2 | **-0.6** | **-0.5** |
| UL-DFL vs. $\mathrm{UL}_\lambda$, both coarse (first version) | **-0.8** | **-1.2** | **-1.0** | **-0.8** | -0.1 | **-1.0** | **-1.1** | **-0.9** | **-0.5** | -0.4 | **-0.8** | **-0.7** | **-0.5** |
| UL-DFL from coarse rule vs. $\mathrm{UL}_\lambda$ fine | -0.3 | **-1.2** | **-1.1** | **-0.5** | **+0.4** | **-1.0** | **-0.4** | -0.3 | +0.1 | -0.0 | **-0.6** | -0.1 | +0.0 |
| UL-DFL vs. $\mathrm{UL}_\lambda$, both fine (reported) | **-0.8** | **-1.2** | **-1.0** | **-1.1** | -0.3 | **-1.0** | **-0.8** | **-0.8** | -0.3 | -0.0 | **-0.8** | **-0.8** | **+0.6** |
| ScenPO-SS, Jia et al. vs. band sampling | **+2.2** | +1.7 | **+3.5** | **-1.8** | **+7.6** | **+5.0** | **+5.0** | +0.5 | **+10.6** | **+3.5** | **+6.3** | +0.3 | **+3.6** |
| $\mathrm{UL}_\lambda$ as published vs. $\mathrm{UL}_\lambda$ (reported) | **+1.7** | **+0.8** | **+0.6** | **+0.6** | **+2.4** | **+1.5** | **+3.4** | **+1.3** | **+1.4** | **+0.9** | **+1.5** | **+1.9** | **+2.9** |
| UL-DFL vs. $\mathrm{UL}_\lambda$ as published | **-2.4** | **-2.0** | **-1.6** | **-1.7** | **-2.6** | **-2.4** | **-3.8** | **-2.0** | **-1.6** | **-0.9** | **-2.2** | **-2.6** | **-2.1** |
| Fill rate (%), ScenPO-SS, Jia et al. | 92.4 | 92.7 | 92.0 | 92.1 | 92.4 | 88.0 | 94.7 | 91.9 | 91.5 | 87.2 | 91.1 | 92.3 | 89.2 |
| Fill rate (%), ScenPO-SS, band sampling | 95.6 | 96.1 | 95.7 | 95.0 | 95.7 | 92.0 | 97.1 | 93.8 | 96.0 | 92.0 | 94.2 | 95.5 | 93.1 |